Rocket GNC Monitor v0a

Operator manual and developer reference
First release: 0.1.1

MIT Rocket Team
Avionics and GNC

25 October 2026

Document control and reading guide

This report documents the released **Rocket GNC Monitor v0a, version 0.1.1**. Part I describes operation at the Launch station and four Away stations. Part II describes the implementation, wire contracts, storage, graphics, extension points and native build process. The appendices contain complete mission, normalized sample and legacy CSV field references.

Item	Reviewed baseline
Application repository	MIT-Rocket-Team/RT_Telemetry_UI; standalone application root after migration.
Release	v0.1.1 / Rocket GNC Monitor v0a, published 25 October 2026.
Source revision	`1923244f1a962049d6f66958816e009edd459e21`. Source links in this report are pinned to the release tag; exact reviewed file hashes are in `evidence/source-manifest.json`.
Report location	Parent project repository, `ui_work/docs/v0a`. The preceding `docs/v0` manual remains a historical record. The application itself is maintained in its standalone repository.
Distributions	macOS universal DMG, macOS 14+; Windows x64 ZIP. Both embed OpenRocket-MIT v6.2, Python, Qt, FFmpeg and Java. Mac uses ad-hoc signatures without notarization; Windows is unsigned.
Validation	Mac ARM: 578 tests passed. Intel: 577 applicable cases covered under Rosetta. Windows: 567 passed, 11 platform skips. Twenty packaged checks per target. See Section 29 for provenance and limits.
Style	KOMA-Script, Latin Modern, A4, blue headings, shaded tables and editable TikZ drawings, following the supplied GenMAPS report style. No raster diagrams.

Release scope This is an implementation reference, not a promise that every planned hardware interface exists. Launch uplink enable, Iris target switching, remote station transport, LTU metrics and sensor fusion remain explicitly incomplete. Local reception, recording, replay, nominal simulation and Away pointing are documented according to their implemented behavior.

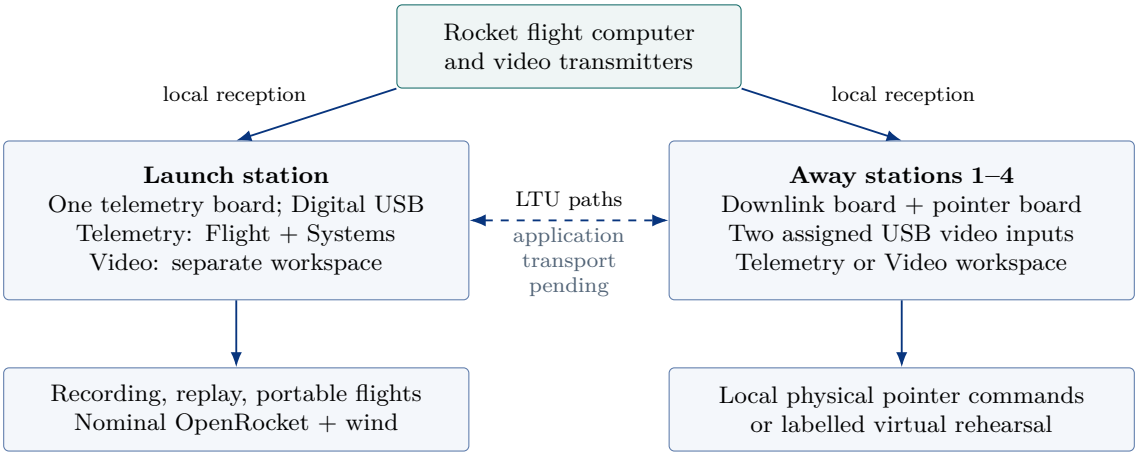


Figure 1: Current station boundary. USB reception is implemented locally. Dashed LTU transfer denotes planned application integration; the existing one-peer snapshot service is separate. Launch uplink enable is a firmware placeholder.

What changed since v0

Station workspaces replace the tab-centered layout: Launch Telemetry spans Flight and Systems, Away Telemetry uses one window, and Video has its own role. The wizard adds numbered stations, optional remembered setup and URRG presets. Iris receiver assignments distribute three system video channels across Away stations. The new rocket graphic distinguishes raw gyro integrals from genuine orientation. The migrated application now has verified Mac universal and Windows x64 packages; physical equipment acceptance remains separate.

Contents

Document control and reading guide **2**

I. Application functionality and operation 5

1. Installation, startup and station identity **5**
1.1. Install the released package . . . 5
1.2. The four-page launch wizard . . . 5
1.3. Command-line selection . . . 5

2. Workspaces and information placement **6**
2.1. Launch Telemetry: Flight and Systems . . . 6
2.2. Away Telemetry and Video roles . . . 7
2.3. State-machine badge and telemetry meaning . . . 7

3. Connections, source isolation and command authority **7**
3.1. Local serial operation . . . 7
3.2. Computer connection versus rocket reception . . . 7
3.3. Launch uplink and Iris selector placeholders . . . 7
3.4. LIVE, DEMO and REPLAY . . . 8

4. Mission, locations and time **8**
4.1. Launch and antenna coordinate entry . . . 8
4.2. URRG presets . . . 9
4.3. Time and flight alignment . . . 9

5. Rocket display, trajectory and GNC **9**
5.1. Main and detailed telemetry . . . 9
5.2. Compact rocket orientation . . . 9
5.3. Full 3D flight viewer . . . 10

6. Away antenna pointer **10**
6.1. Model and manual operation . . . 10
6.2. Physical tracking . . . 11
6.3. Virtual pointer and trajectory rehearsal . . . 11

7. Video roles and routing **11**
7.1. Capture and device exclusivity . . . 12
7.2. Launch video wall . . . 12
7.3. Recording and replay . . . 12

8. Legacy rocket commands, power and recovery **13**

9. Wind and nominal OpenRocket reference **13**
9.1. Wind preparation . . . 13
9.2. Simulation and import . . . 13

10. Recorded test-launch demonstration **13**

11. Recording, sessions and portable flights **14**
11.1. Capture and session folders . . . 14
11.2. One-file save/load . . . 14

12. Station networking and fusion status **15**
12.1. Existing one-peer LAN . . . 15
12.2. APRS recovery information . . . 15

13. Settings, shortcuts and practical checks **15**
13.1. Global Settings . . . 15
13.2. Preserved and added keyboard shortcuts . . . 16
13.3. Operating sequence . . . 16

II. Codebase and developer documentation 17

14. Repository, entry points and source ownership **17**
14.1. Module map . . . 17
14.2. Dependencies and resource resolution . . . 19

15. Runtime architecture and state ownership **19**
15.1. Concurrency model . . . 19
15.2. Connection generations and asynchronous results . . . 20
15.3. Mode transitions . . . 20
15.4. Lifecycle and shutdown . . . 21

16. Station profiles, routing and future transport **21**
16.1. Startup persistence and role boundaries . . . 21
16.2. One Launch board, Away routes and placeholder state . . . 21
16.3. Video-wall integration contract . . . 21
16.4. Existing networking and the fusion boundary . . . 22

17. Rocket orientation and flight-effect contracts **22**

18. Data contracts, units and geographic frames **22**
18.1. Mission and Sample . . . 22
18.2. WGS84 and local ENU . . . 23
18.3. Time normalization and reboot behavior . . . 23

19. Serial and protocol implementation **24**
19.1. Port ownership and transport rules . . . 24
19.2. Telemetry framing and checksum . . . 24

19.3. Pointer command packets . . . 25
19.4. Rocket uplink packets . . . 25
20. Recording, replay and portable archive internals **26**
20.1. Session writer and durability . . . 26
20.2. Raw capture format . . . 26
20.3. Replay reader . . . 26
20.4. Flight archive layout . . . 27
20.5. Active snapshot consistency . . . 27
20.6. Archive validation and application . . . 28
20.7. Read-only analysis example . . . 28
21. Legacy import and demo provenance **28**
22. Video implementation and synchronization **29**
23. Weather and OpenRocket bridge contracts **29**
23.1. Weather normalization . . . 29
23.2. Trajectory contract . . . 29
23.3. Simulation job boundary . . . 30
23.4. Reproducibility and interpretation . . . 30
24. UI, font and antenna graphics maintenance **31**
24.1. UI responsibilities . . . 31
24.2. Where the antenna's physical description is defined . . . 31
24.3. Changing the model without changing hardware behavior . . . 31
25. Smooth animation and 3D flight rendering **32**
26. Virtual pointer transport and trajectory geometry **32**
27. Settings storage and application **33**
28. Developer setup and native packaging **33**
28.1. Prepare a standalone checkout . . . 33
28.2. Engine and private Java runtime . . . 34
28.3. Build with or without tests . . . 34
28.4. Universal Mac composition . . . 35
28.5. Verification, signing and cleanup . . . 35
28.6. Manual Windows CI . . . 35
29. Testing and release evidence **35**
29.1. Regression responsibilities . . . 35
29.2. 0.1.1 validation baseline . . . 36
30. Extension and maintenance guidance **37**
30.1. Hardware and new telemetry . . . 37
30.2. Data, simulation and presets . . . 37
30.3. Known follow-up work . . . 37
A. Complete mission configuration reference **38**
B. Complete normalized sample reference **39**
C. Legacy CSV columns and derived values **40**
D. Worked file examples **41**
D.1. URRG planning template . . . 41
D.2. Wind profile . . . 42
D.3. Normalized trajectory . . . 42
D.4. Portable flight metadata excerpt . . . 42
E. Startup preferences example **43**
F. Source and validation index **43**
G. Glossary **44**

Part I.
Application functionality and operation

1. Installation, startup and station identity

The monitor combines rocket telemetry, Away antenna pointing, local USB video, recording/replay and a nominal OpenRocket reference. It runs offline for local devices, saved flights, demos, coordinates and prepared simulations. Internet access is needed for online weather and initial build downloads. The monitor does not require internet for its existing direct station LAN connection.

1.1. Install the released package

Download the platform asset and optional SHA-256 sidecar from the 0.1.1 release. On macOS 14 or newer, open the universal DMG and copy the complete application to Applications. It selects a native Apple Silicon or Intel payload. The release is ad-hoc signed and not notarized; if macOS blocks first launch, use the operating system's explicit approval for the trusted downloaded app. On Windows, extract the entire x64 ZIP and run the executable within its folder. Keep `_internal` and all adjacent files together. Windows ARM runs this x64 distribution through emulation. Linux is not a validated release target.

Python, Qt, FFmpeg, Java and OpenRocket are included. Recipients do not install development tools. USB drivers and camera permissions depend on the computer; Dire Wolf is a separate optional dependency for APRS. Lucida Grande is loaded privately from the package without installing a system font.

1.2. The four-page launch wizard

Choose **Launch station** or **Away 1–4**, then **Telemetry** or **Video**, **Balius** or **Iris**, and **URRG** or a custom site. Station identity and role remain fixed for this session. Finish opens LIVE with serial devices disconnected and cameras stopped. Cancel exits before a device controller is constructed.

Remember these choices next time is off by default. Without it, startup defaults are Launch / Telemetry / Balius / Custom, subject to CLI overrides. With it, the wizard is prefilled from the saved choices. Uncheck it and finish to stop remembering. Cancel preserves the previous remembering preference. **File → Forget setup on next launch** clears only startup choices, has no keyboard shortcut, and does not alter the running mission or global settings.

New missions are named **Balius Launch** or **Iris Launch**. URRG fills the pad and applicable station coordinates. Elevations remain separate entries. Opening a saved mission or flight restores that file's name and locations; startup defaults do not replace them.

1.3. Command-line selection

From the standalone source repository:

```
.venv/bin/python -m rocket_gnc_monitor \
  --site away3 --role telemetry --vehicle iris \
  --launch-site urrg --skip-setup
```

The packaged executable accepts the same arguments. On Mac it resides inside `RocketGNCMonitor-v0a.app/Contents/MacOS/RocketGNCMonitor-v0a`; keep the complete bundle in place. On Windows invoke the extracted executable.

Option	Behavior
`-site`	`launch`, `away1`, `away2`, `away3`, `away4`. The old `base` spelling is a compatibility alias.
`-role`	`telemetry` or `video`.
`-vehicle`	`balius` or `iris`.
`-launch-site`	`urrg` or `custom`.
`-skip-setup`	Start directly with resolved choices; otherwise review them in the wizard.

Option	Behavior
`-remember-setup`	Check the opt-in; with skipped setup, save the resolved choices.
`-no-remember-setup`	Clear the opt-in; with skipped setup, remove remembered choices. Without either remember flag, skipped setup leaves saved choices unchanged.
`-station`	Legacy shortcut: `launch`, `away`, `video` preselect Launch Telemetry, Away 1 Telemetry, Away 1 Video. Does not bypass setup.
`-demo`	Explicitly request recorded DEMO instead of normal LIVE startup.
`-demo-station`	`GS1`, `GS2`, `GS3`; default GS2.
`-data-dir`	Separate per-user application data/preferences directory.

2. Workspaces and information placement

Station role and source mode are independent. The same telemetry workspace can show LIVE, DEMO or REPLAY. None of those source-mode names is the rocket's onboard state. Video does not occupy either telemetry workspace.

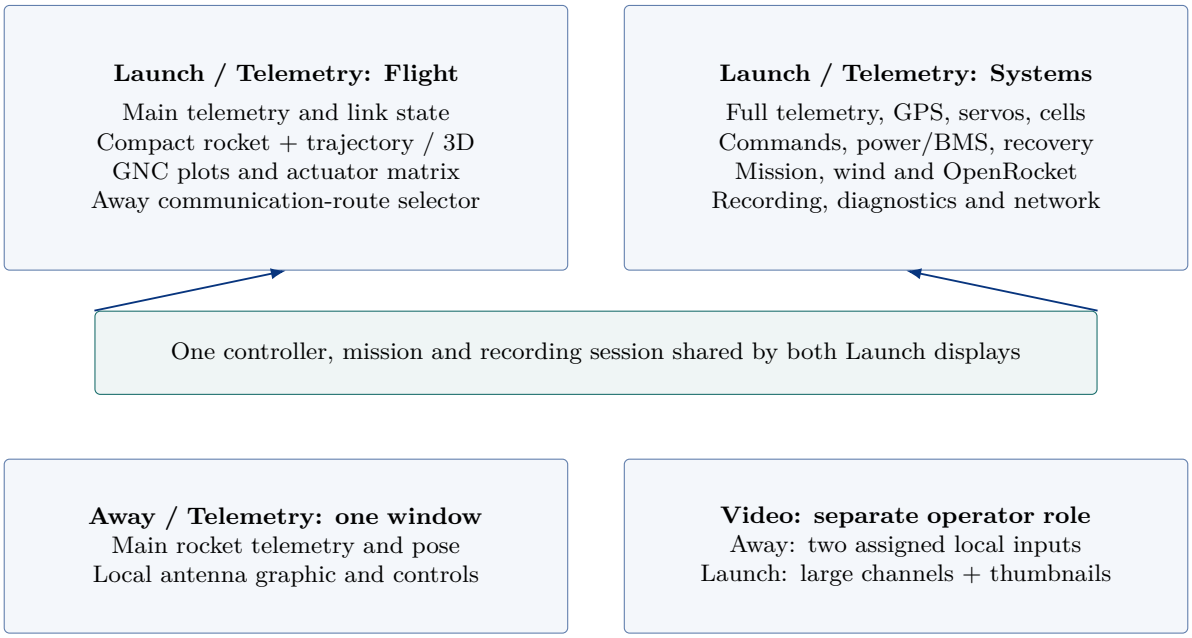


Figure 2: Information placement in v0a. These are functional layouts, not pixel-exact screenshots. Configuration and file dialogs remain deliberate actions.

2.1. Launch Telemetry: Flight and Systems

Flight shows primary telemetry, state/freshness, the ground-board and rocket-link indicators, flight plots/reference, the full 3D viewer, compact rocket orientation display, GNC plots and actuator channels. Its communication panel selects a desired route through Away 1–4. Launch has no local physical or virtual antenna pointer.

Systems shows the complete legacy telemetry and GPS tables, servos, cells, rocket commands, power/BMS and recovery controls, mission/wind/OpenRocket, sessions/replay, diagnostics and APRS/network. A large state badge occupies the upper-right corner. Both windows share one controller, mission and recording session. There are no duplicated serial workers.

The intended layout is two 1920 by 1080 displays, approximately 1920 by 1020 usable window space. **View** → **Arrange station displays** positions the windows on available screens. Both remain accessible on one screen for preparation. Closing Flight exits the workspace; the Systems close button keeps that required display open. Long diagnostics, JSON and wind profiles scroll within visible panels. Smaller windows/display scaling can crowd controls; this release does not resolve every compact-layout issue.

2.2. Away Telemetry and Video roles

Away Telemetry places main rocket telemetry, the large compact rocket and its altitude plot beside the local articulated pointer and controls. Advanced mission, settings and file operations remain available through the menus and dialogs. Its downlink and pointer boards connect independently.

Away Video presents the two USB inputs assigned to that station. Launch Video presents two Balius or three Iris large channels with eight Away thumbnails below. Video roles hide serial controls and rocket instruments, retain camera, recording/replay and settings controls, and need no telemetry-board connection. See Section 7 for exact receiver assignments.

2.3. State-machine badge and telemetry meaning

The large Systems badge follows the received state, including unexpected transitions in recorded data. It never takes its value from a simulation.

State	Color	Interpretation
`GROUND_TESTING`	Green	Received ground-testing state.
`PREFLIGHT`	Blue	Received preflight state.
`FLIGHT`	Red	Received flight state.
`POST_APOGEE`, `MAIN`, `END`	Orange	Recovery/post-apogee or end state.
Unknown or unavailable	Black	Missing telemetry or an unrecognized state.

The caption identifies LIVE, DEMO or REPLAY. Stale LIVE reception retains the last state with an explicit **STALE** annotation. A state color does not indicate command authorization, verified liftoff, motor burn or deployment.

3. Connections, source isolation and command authority

3.1. Local serial operation

Launch has **one telemetry board**, used for downlink and intended Launch-only uplink. Away has one downlink board and a separate pointer board. Both existing serial protocols use 115200 baud, 8 data bits, no parity, one stop bit, no flow control and the previous monitor's default DTR/RTS behavior. Select the physical device and connect explicitly. Refresh after plugging in equipment. A connected or connecting port disappears from the other board's dropdown, including macOS tty/cu aliases, and returns after disconnection.

Connecting the telemetry board opens its port. **Start Polling** separately enables reception; **Stop Polling** leaves the port open. The original ground read timeout is one second; pointer reads use 0.1 second. No initialization command is automatically sent on connection or polling start. Reconnect is manual. Tracking never resumes automatically after stale data or disconnection.

3.2. Computer connection versus rocket reception

The link panel distinguishes disconnected, connected with polling stopped, waiting for telemetry, live reception, and lost/stale reception. USB connected means that a port opened. Rocket live means a checksum-valid LIVE packet arrived within the mission freshness limit. There is no separate radio handshake. Reconnection requires a new packet before reporting live radio reception. Rejected packets do not update accepted telemetry or replay data.

3.3. Launch uplink and Iris selector placeholders

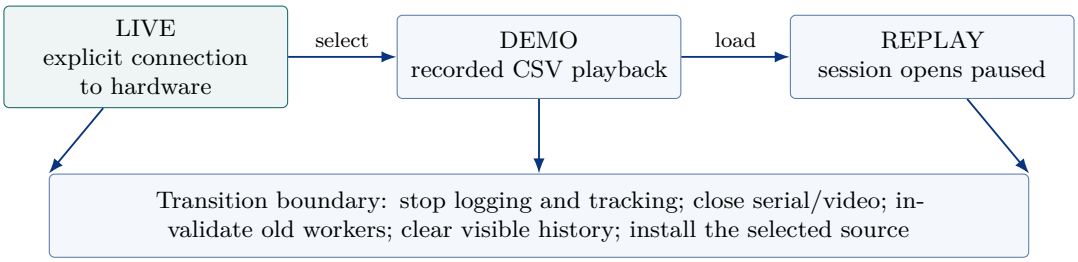
The Launch board's software uplink-enable command is not defined. LIVE rocket commands therefore remain locked for Balius and Iris. The DEMO-only switch simulates enable/disable and logs events. Away stations have no uplink authority. Selecting a remote Away route or receiving a peer snapshot cannot unlock it.

For Iris the same Launch board has one Sustainer/Booster selection shared by downlink and uplink. Away 1–3 initially desire Sustainer, Away 4 Booster. The switching command and acknowledgement are also undefined. Selectors are labelled **PLACEHOLDER**; LIVE/REPLAY switching is unavailable and physical target state is

unknown. **Simulate switch** in DEMO changes the indicated simulated state without replacing or relabeling the Zephyrus recording. Leaving DEMO clears this state. No fabricated selector or uplink-enable bytes are transmitted.

3.4. LIVE, DEMO and REPLAY

LIVE starts disconnected. DEMO runs the recorded test flight and simulates commands. REPLAY reads a saved session and cannot transmit. Mode changes close transports, stop logging/tracking and clear incompatible history; DEMO and REPLAY cannot construct physical serial workers. Virtual-pointer rehearsal is an explicitly labelled port-free device in Away LIVE, independent of radio readiness.



A planning-only flight returns to disconnected LIVE. A session-bearing flight restores paused REPLAY. Source-mode names are independent of the rocket's onboard flight state.

Figure 3: Mode isolation. Other mode selections use the same transition boundary; the upper arrows illustrate common operator paths rather than the only permitted transitions.

4. Mission, locations and time

4.1. Launch and antenna coordinate entry

Mission → **Configure** accepts launch latitude/longitude, MGRS or Plus Code, all resolved offline to WGS84. MGRS accepts spaces/lowercase, UTM/polar grids and displays precision; its decoded grid reference point is used. Full Plus Codes resolve to the area's center. Short codes require nearby reference latitude/longitude; locality names alone are not geocoded. Review the resolved coordinates before saving. Switching display formats alone keeps canonical coordinates; editing them changes a named preset to Custom.

Enter ellipsoid altitude for geographic transforms and MSL elevation for weather/atmosphere separately, and establish the launch origin. The legacy height interpretation may be unknown, ellipsoid GPS, GPS relative to launch, or barometric relative to launch. Select the interpretation verified for the firmware. This controls trajectory comparison, while physical legacy pointing retains its separate original convention. Stop recording before mission edits.

At Away stations, antenna entry accepts lat/lon, MGRS or **Relative to launch**. Relative inputs are true heading *from antenna toward the rocket on its launch pad*, horizontal range 0–100,000 m, and antenna altitude minus launch altitude. Heading 90 degrees, distance 500 m, height +5 m places the antenna west of the pad and 5 m above it. Relative entry recalculates when the launch origin changes; absolute entry retains its coordinates. Confirm **Antenna location established** for virtual trajectory tracking.

4.2. URRG presets

Location	MGRS
Rocket launch pad	18TUN2061530290
Launch station reference	18TUN2063730181
Away 1 antenna	18TUN2177730106
Away 2 antenna	18TUN2291333237
Away 3 antenna	18TUN2015031101
Away 4 antenna	18TUN2259229678

Wizard selection fills pad and applicable Away antenna coordinates. The Launch station's reference does not create a local pointer. Mission antenna presets preserve the entered antenna altitude. None supplies surveyed elevation, calibrates mechanics or sends a command. Saved missions/flights preserve their own preset labels, exact codes and canonical coordinates.

4.3. Time and flight alignment

Device time is uptime, with rollover extension; it is distinct from host receive time, recording elapsed time, UTC and simulation time. Flight zero is established by an observed Preflight-to-Flight transition or explicit operator alignment. Joining mid-flight leaves it unaligned. A detected reboot invalidates alignment; realign deliberately. DEMO uses each receiver's own recorded timing. Replay uses host elapsed time and restores alignment events. Video synchronization is host timing with a configurable offset, not camera exposure synchronization.

5. Rocket display, trajectory and GNC

5.1. Main and detailed telemetry

The overview shows available altitude, velocity, GPS, radio/packet freshness, phase and flight-time information. Systems adds GPS accuracy/satellite counts, maximum altitudes, temperature, packet number, legacy servo drives and angles, three battery cells, power/BMS and pyro values. Missing values stay unavailable. GNC plots preserve the reported gyro rates and integrated rotations; new canard/fin-tab feedback is not inferred from the four legacy servo channels.

Track coordinates are launch-relative east/north/up with an explicit origin and altitude interpretation. A simulation reference is a separate dashed path and same-time marker; it does not replace measurements. DEMO uses its own labelled local display frame. Alerts and diagnostics expose accepted/rejected packet counts, discarded bytes, inferred gaps, full samples and events. Low-battery alerting uses a one-second dwell and clears 0.3 V above the threshold. Acknowledgement records an event without erasing an active condition.

5.2. Compact rocket orientation

The large model replaces the circular attitude indicator; readouts and status are underneath. The **View azimuth** slider changes camera direction from 0 to 360 degrees while vertical stays vertical. This pane has no drag, pan, scroll or hardware-control interaction.

Orientation source Zephyrus roll/pitch/yaw values are cumulative body-axis gyro integrals, not Euler attitude. Their numerical readouts remain, but the model is neutral and muted with **ORIENTATION UNAVAILABLE**. Its upright placeholder does not assert a measured vertical attitude. Flight phase or climbing altitude cannot establish orientation.

A future adapter can supply an explicitly validated body-to-ENU quaternion. Current telemetry also lacks confirmed motor-burning and parachute-deployed fields, so those effects are unknown. Pyro fired and flight state do not prove deployment; historical ignition does not prove continued burning.

Choose **OpenRocket reference** explicitly for simulated orientation and events. This focuses the embedded Launch 3D controls or opens the Away 3D window. Playback, scrub and event jumps drive the reference pose. The pane labels reference time and playback state. A simulated burn interval displays a flame; a recorded simulated deployment displays a canopy. Switching back to Telemetry restores its independent values and availability.

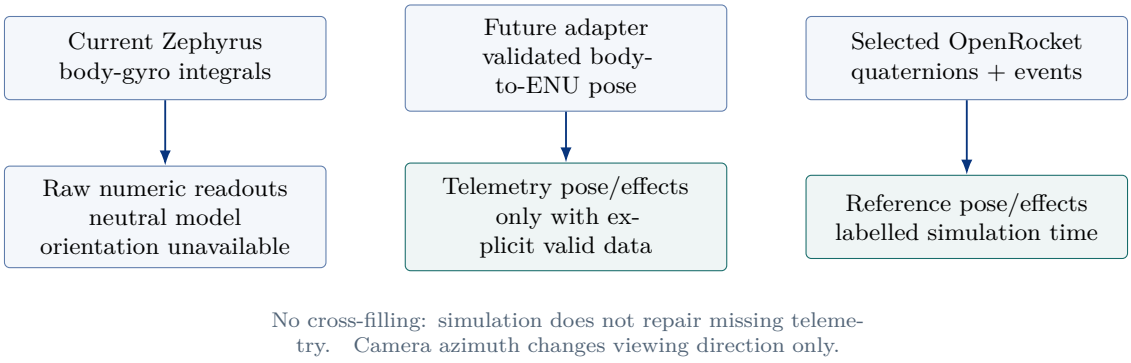


Figure 4: Orientation provenance. Phase, increasing altitude and pyro bits do not supply missing orientation, motor-burning or parachute-deployment evidence.

5.3. Full 3D flight viewer

Use **View** → **3D flight** after loading or simulating a reference. Drag to orbit, Shift/right-drag to pan, scroll to zoom and double-click or Fit to reset. Perspective/Top/Side views, follow-rocket, display-size, velocity and ground projection controls affect viewing only. Play, rewind, speed, timeline and event jumps inspect ignition, burnout, apogee, deployment and touchdown when those events exist. **Follow monitor time** uses aligned flight time or virtual rehearsal time; turn it off for independent playback.

Gold denotes reference, teal actual track when origins agree, purple antenna. Old four-column references remain usable with an explicitly path-aligned illustration and unavailable attitude/events. Tumble/recovery can leave engine-held attitude; undersampled rotation is labelled. Missing events are never inferred from descending motion. The rocket, plume and canopy are schematics, not CAD or deployment dynamics. The Zephyrus model has no recovery event in its nominal output; the bundled small example supplies one.

6. Away antenna pointer

6.1. Model and manual operation

The four-leg stand supports an azimuth turntable and an elevation carrier. Grid dish, long Yagi and enclosed Avenger XR18 attach to the common beam through the support pivots. The dish-colored feed arm is collinear with the boresight. Dimensions and hidden mechanics are illustrative. Drag to orbit from any angle, scroll to zoom, and double-click or choose a preset to reset. Camera movement is independent of command authorization.

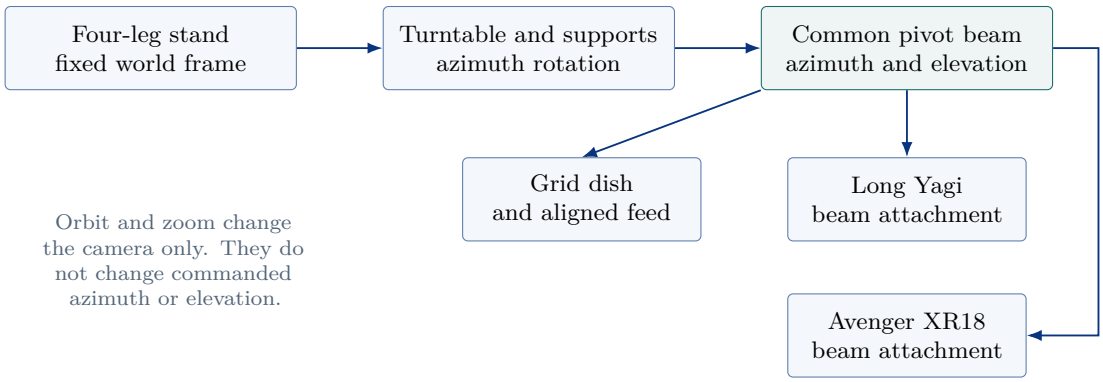


Figure 5: Transform hierarchy of the procedural antenna model. Both outer antennas and the reflector belong to the same moving carrier; this diagram describes attachments, not dimensions or a photographic viewing direction.

Enter azimuth/elevation and press Send. Editing a field alone does not move or transmit. UP/DOWN/LEFT-/RIGHT send the original direction opcodes; ZERO resets software counts at the physical current position, rather

than homing to a switch. Manual controls require only the pointer connection, independently of telemetry. A direction command before a known target leaves angles unknown.

Sent means serial dispatch, not acknowledgement or measured arrival. The graphic smoothly approaches the last target on its independent animation timer. Smoothing changes no commands or timing. Measured pose is unavailable in the existing pointer protocol.

6.2. Physical tracking

Connect both boards, start polling and use **Send to AntPtr** to freeze ground receiver GPS. **Start tracking** sends targets at up to 5 Hz using rocket GPS lat/lon and filtered barometric height. It preserves receiver lat/lon rounded to five decimals, receiver height fixed to zero for pointing, and the historical displayed receiver-height division by 1000. Mission trajectory datum settings do not alter this legacy path. Unfreeze updates receiver position. Stale/invalid telemetry or disconnection stops host tracking; restart explicitly.

Stopping host tracking Stop tracking removes unsent work and stops new targets. The last commanded physical move may continue. The firmware has no stop/watchdog, measured pose, mechanical homing or acknowledgement. Use the equipment's independent stop/power control to interrupt physical movement.

6.3. Virtual pointer and trajectory rehearsal

In Away LIVE, configure an established antenna location and load a georeferenced reference. Select **Virtual antenna pointer**, connect, and use the same manual commands or **Follow trajectory**. No telemetry connection is needed. The simulation slews at illustrative maximum rates of 90 degrees/s azimuth and 60 degrees/s elevation. Playback has pause/resume, rewind, time scrub and 0.25–4 times speed. Manual commands pause rehearsal; Hold freezes its clock and motion. Changing reference/location stops it and requires restart.

The banner, pose and connection are labelled VIRTUAL/SIMULATED. A gold target and purple antenna marker remain distinct from telemetry. The stored reference origin and antenna ellipsoid position define the direction. Coincident points have no direction; exact zenith retains the previous azimuth. Rehearsal neither opens a serial port nor creates telemetry or unlocks rocket controls. Changing source mode closes it; flight loads do not restore a simulated connection.

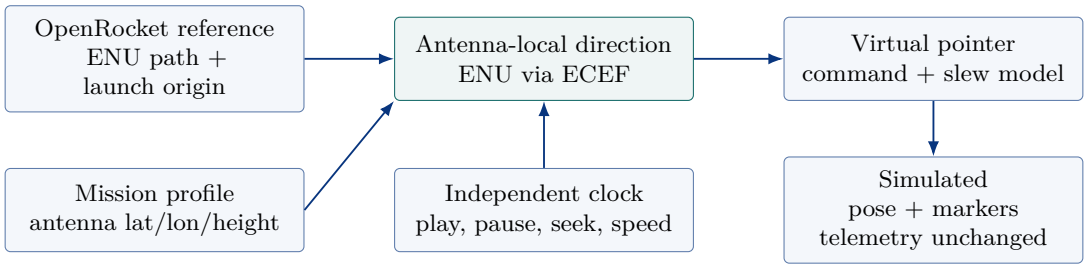


Figure 6: Away virtual rehearsal uses the saved antenna position and its own clock. No physical serial transport participates in this path.

7. Video roles and routing

Digital/Analog labels describe the rocket-to-ground RF paths. All locally selected inputs are USB camera captures. Telemetry is not required to start video or a video-only recording.

Profile	Local receivers	Large Launch channels / source assignment
Balius Away 1–4	Digital + Analog	Every Away contributes one of each.
Balius Launch	Digital only	Digital initially local; Analog initially Away 1.
Iris Away 1–3	Sustainer Digital + Sustainer Analog	Sustainer telemetry/APRS assignment.
Iris Away 4	Sustainer Digital + Booster Analog	Booster telemetry/APRS assignment.
Iris Launch	Sustainer Digital only	Digital initially local; Sustainer Analog initially Away 1; Booster Analog initially Away 4.

Iris has three system channels but *two local USB feeds per Away station*. The later receiver-assignment diagram refines the earlier three-local-camera proposal. Stage names are assignments, not decoded identification from legacy packets. Launch's one telemetry board targets one stage at a time.

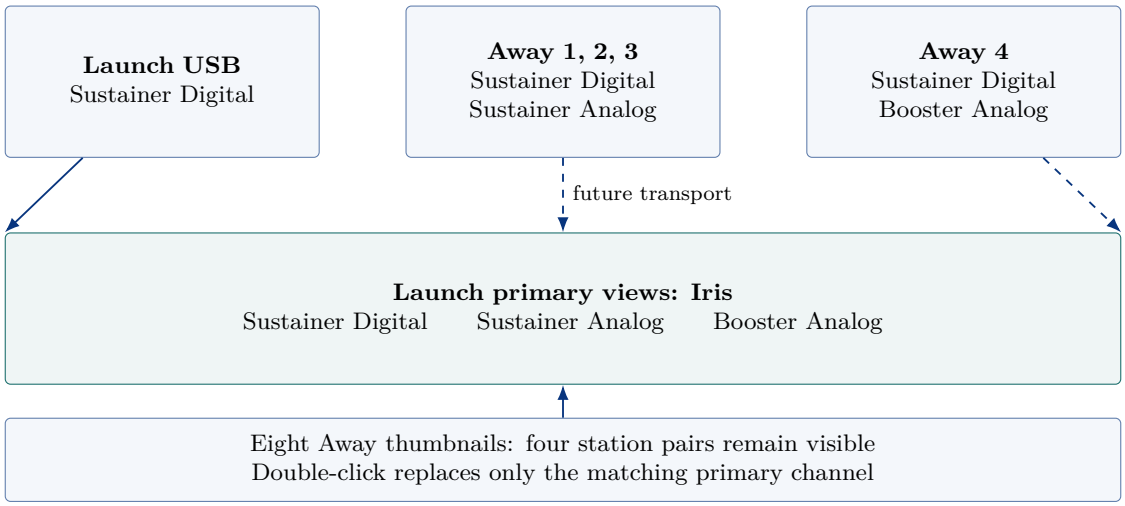


Figure 7: Iris has three system channels and two local inputs per Away station. For Balius every Away pair is Digital/Analog and Launch has two primary views. Dashed paths are not implemented network feeds.

7.1. Capture and device exclusivity

For each input choose Find, select a USB receiver and Start. Stop closes only that channel; File selects local video. Each panel reports frame age and faults. Grant OS camera permission when required. A selected camera disappears from other local selectors; active and closing cameras remain reserved. Clear an unused selection to release it. One failed/reconfigured stream does not stop another. These reservations apply within the application's local controllers, not as a cross-process camera arbitration service.

7.2. Launch video wall

All four Away pairs remain visible as thumbnails, including lower-quality alternatives. Double-click one to replace *only its matching large channel*. Away 4 Booster Analog replaces Booster Analog; Digital always replaces Digital. **Use local Digital** restores the Launch USB feed without having closed its capture during promotion.

Remote slots currently show **Awaiting station link**. Choosing a source does not establish a link. The presentation interface accepts future frames, quality text, receive age and loss/stale states; five seconds without fresh remote frames becomes stale. Interstation video transport and remote recording are not implemented. The existing telemetry LAN link carries no video.

7.3. Recording and replay

After local frames arrive, Start recording creates a session independently of serial telemetry. Channels have separate FFmpeg segments and timing. DEMO video is explicitly a TEST PATTERN, not Zephyrus flight footage. REPLAY exposes flight/session load, play/pause, seek and speed in the simplified footer. Launch replays its local Digital channel; inactive analog assets remain in the archive. Remote frames are display-only in the present integration API.

8. Legacy rocket commands, power and recovery

Systems retains Advance State, Zero P/Y/Roll, Zero Altitude, Zero Velocity, Zero Servos, PD Activate, roll/airbrake angle entry and VTX 1/3/5/8 W. Six power-rail requests/readbacks cover 3, 3.3, 5, 7.4, 8.4 and 28 V, with current, voltage, total and temperature. The 3.3 V request remains on; Total is an aggregate indicator. BMS displays all eight enabled/triggered protection bits.

Recovery has six channels with continuity, arm/fire status and resistance. Select channels 0–5 for ARM/FIRE. State advance, ARM/FIRE and emergency piston, BP wells, tender descender and all-recovery actions retain their confirmation dialogs and original packet encoding. These controls are present for parity; normal Launch LIVE dispatch remains locked pending the uplink switch contract. In DEMO they can be exercised after simulated uplink enable. Neither a command event nor a fired bit proves a physical recovery outcome.

9. Wind and nominal OpenRocket reference

9.1. Wind preparation

Enter/import layers or retrieve online weather for a specified site and a timezone-bearing time, e.g. `2026-10-25T16:00:00Z`. Apply after table edits. Heights are above launch in metres, speed is m/s and direction is meteorological *from* true north. Internal layers store east/north velocity components. Layers must have increasing nonnegative height; interpolation holds endpoint wind beyond coverage.

The weather adapter uses the nearest available hour within one hour and converts geopotential height to geometric MSL height, then subtracts launch MSL elevation. Review available altitude/time coverage. Requests/responses are retained for offline reuse. Weather is attributed to Open-Meteo, CC BY 4.0. Manual profiles remain available without network access.

9.2. Simulation and import

Select an ORK with a saved simulation/motor configuration. Simulation index zero selects the first saved simulation. The bundled MITRT N8406 curve resolves the supplied Zephyrus test-launch model; other custom ENG/RSE curves can be added. Missing motors are reported by name. Run simulation in a separate job; Cancel stops it without interrupting telemetry. Inspect the job's request, copied inputs, hashes, trajectory metadata and worker log on failure.

The bridge uses the selected saved motor/flight configuration, configured site/rail/wind, ISA atmosphere, fixed seed, zero turbulence standard deviation and first output branch. It disables saved extensions and the fork's global inertia override. Branch count and warnings are recorded. Controlled three-axis canard/tab dynamics remain unqualified; this is a nominal reference.

Imported references use CSV `time_s,east_m,north_m,up_m` with matching JSON declaring schema 1, ENU, units `m,s` and geographic origin `[latitude,longitude,ellipsoid_altitude]`. Up is launch-relative. LIVE requires compatible origin and nonsynthetic data. References are not extrapolated outside their time coverage. Optional quaternion, rate, velocity and event data support 3D playback; old position-only references remain usable.

10. Recorded test-launch demonstration

Select DEMO explicitly. GS2 is the default; GS1 and GS3 are independently selectable. The bundled gzip files reproduce the supplied CSV bytes offline. These are three receivers of the same launch, not successive flight segments and not a fused dataset.

Receiver	Rows	Receive duration	Notes
GS1	4,366	274.056 s	GPS corruption present.
GS2	5,464	330.234 s	Default, greatest coverage.
GS3	4,939	332.725 s	No valid receiver GPS.

Playback starts five receive-time seconds before the first FLIGHT packet, device time 747.766 s. Play/pause, seek, 0.25–4 times speed, Launch −5 s and Full start control playback. Gaps, original UTC/packet numbers and values

are preserved. Seeking reconstructs prior history. Stop logging before changing recording or seeking; pause/speed changes are allowed during logging.

All logs end during MAIN descent, with no touchdown. They include an approximately 8.4-second reception gap, absent fixes, zero cell readings and gyro saturation. GS3 briefly reports Preflight during descent. The app retains these values, rather than correcting flight history. Track-only checks omit missing 3D fixes and points over 20 km from launch; the banner identifies them. The display uses GPS horizontal offsets and reported barometric height, with no surveyed-datum claim. GS3 cannot rehearse automatic receiver-GPS tracking; GS1/GS2 can. Manual simulated controls remain available.

No raw frames or launch video were supplied. CSV checksums cannot be revalidated; demo video is a labelled test pattern. Commands are simulated. Returning to LIVE clears demo data and restores the prior mission/reference.

11. Recording, sessions and portable flights

11.1. Capture and session folders

Start Logging immediately creates a unique folder when eligible telemetry or video is available. The default is the app-data sessions folder, overridable in Settings. Diagnostics shows its path. The recorder is independent of redraws; its drop/error/completion status is separate from UI frame drops. Stop Logging finalizes outputs; quitting also closes active workers/recording.

Each session may contain a manifest, indexed SQLite samples/events, CRC-protected raw serial RX/TX, 43-column `telemetry.csv` and rejected-packet CSV, a reference, imported source CSV and per-channel video segments. Opening a session supports play/pause, seek, 0.1-second step and 0.25–4 times speed. Seek removes future state and restores prior alignment and sent-pointer events. CSV export of normalized samples includes full sample JSON and differs from legacy CSV.

Video follows host timing. Positive mission offset delays it; negative advances it. Paused seeking decodes one frame per recorded stream. Stopping a replay feed keeps it paused until the next seek/restart. Old single-stream recordings map to Digital. Reconnect creates a new segment directory. Incomplete sessions can be read when committed SQLite data survives; retain a crashed session's `-wal` file for recovery. A crash can lose the active segment/uncommitted samples without invalidating finalized data.

11.2. One-file save/load

File → Save flight writes a portable `.rktflight`; Open flight restores it. The file includes mission/site/wind/weather, model/custom motors, reference and available recordings. Embedded assets no longer depend on their original paths. Saving is atomic; loading validates inventory, paths, hashes and contents before replacing current state.

Save context	Content and limit
Active logging	Committed snapshot; finalized video segments only. Stop Logging and save again for the closing segment and final samples.
Stopped logging	Last session in the current source mode; waits for video closure.
REPLAY	Opened session and playback position.
DEMO without recording	Entire selected receiver dataset and current position, with no invented raw packets or flight video.
LIVE without recording	Displayed buffer, at most 6,000 samples, labelled with that limited scope.
No telemetry	Planning flight with mission/model/reference.

Stop logging before opening another flight. A session-bearing file opens paused in REPLAY with hardware disconnected. A planning file opens disconnected LIVE. Pointer calibration and connections are not restored. Mission JSON separately stores configuration and can retain external asset paths; use a flight file for portable assets. Extracted flights live in the app-data flight cache.

Legacy CSV import accepts the audited layout, retains original bytes/extra columns and converts to replay. Other CSV layouts are rejected. Imported decoded rows cannot recover radio checksums or establish new vehicle feedback.

12. Station networking and fusion status

Launch's intended topology is Ethernet/PoE to four LTU-XR links, each ending at an Away station with its local WLAN. Away Wi-Fi selection stores a desired SSID; Refresh reads OS current/saved networks and Open Wi-Fi settings opens the OS joining controls. The selected SSID and actual OS connection are distinct. The monitor stores no password and does not join or configure the bridges.

Launch's Away pointer selector records a desired communication route. It does not move a pointer, enable uplink or prove a link. Remote telemetry/video/control transport, LTU signal metrics, analog receiver tuning and interstation calls are pending. No sensor fusion is implemented: local telemetry, peer snapshots, APRS and simulation remain separate sources.

12.1. Existing one-peer LAN

Join the local network, Listen on one computer (default TCP 8765), then Connect from the other to its LAN IP and port. The listener binds local IPv4 interfaces and accepts one peer. About twice per second the two sides exchange bounded, versioned, read-only snapshots of selected telemetry, pointer/link state and available APRS. The panel shows peer name/mode/link, altitude, phase and age. Stop closes the link and removes disconnected peer data. This connection carries no video/audio/commands and does not supply the future four-station transport or a fused navigation solution.

12.2. APRS recovery information

Configure an external Dire Wolf audio decoder and KISS TCP, then connect the APRS row (default `127.0.0.1:8001`). An optional full callsign including SSID filters fixes; blank accepts the latest supported position. The panel reports connection, callsign, coordinates, optional altitude and age. A disconnected last fix stays labelled as such.

Supported AX.25 UI payloads are uncompressed APRS `!`, `=`, `/` and `@` positions. Compressed, Mic-E, objects, NMEA and ambiguous positions are unsupported. `/A=` altitude converts feet to metres without assuming a datum. APRS is receive-only recovery information and cannot replace telemetry or actuate pointing. Automatic Iris identification/relay is pending. Host/port/filter settings persist in `station_network.json`; connections start off. Dire Wolf is neither bundled nor configured by this app.

13. Settings, shortcuts and practical checks

13.1. Global Settings

Open Settings with Command-comma on Mac or Ctrl-comma on Windows. The dialog is available disconnected and in every role. OpenRocket JAR selects a compatible complete team engine; Use bundled restores the packaged engine. Its effective path/version are displayed. Simulation timeout is 10–1800 s, default 120. Recording folder affects future sessions only. Dark/Daylight appearance persists; the View menu toggle shares that preference.

Save validates and stores; Cancel discards edits. Restore Defaults updates the form and takes effect on Save. `settings.json` resides in the displayed per-user data directory; `-data-dir` isolates it. Global preferences are not included in mission or flight files. Running simulations keep their submitted engine/timeout. Missing custom JARs produce an error rather than silently falling back. Startup never auto-connects equipment.

13.2. Preserved and added keyboard shortcuts

Qt maps Ctrl to Command and Alt to Option on macOS, as in the original UI. Actions retain role/connection gating; shortcuts cannot bypass a disabled hardware capability. Launch has no pointer shortcuts to a local device; Video hides serial actions. The ten original serial/pointer key sequences are:

Action	Windows / Qt	macOS
Refresh serial ports	Ctrl+R	Command+R
Telemetry connection toggle	Ctrl+Alt+C	Command+Option+C
Pointer connection toggle	Shift+Ctrl+Alt+C	Shift+Command+Option+C
Start/stop polling	Ctrl+Return	Command+Return
Start/stop logging	Ctrl+L	Command+L
Pointer UP/DOWN/RIGHT/LEFT	Ctrl+arrow	Command+arrow
Pointer ZERO	Ctrl+0	Command+0

Open/Save flight add Ctrl+O/Ctrl+S (Command on Mac); Settings adds Ctrl+comma (Command on Mac). Forget setup deliberately has no shortcut. Serial actions are bound once and work from either Launch display.

13.3. Operating sequence

Choose station/role/vehicle/site; review elevations and mission height datum. Connect the appropriate local inputs explicitly. For telemetry, start polling and distinguish USB readiness from recent rocket reception. For video, check each channel label and independent input. Start logging before the interval that must be retained. On Away, verify pointer orientation and ground-GPS freeze before physical tracking, or use explicitly virtual rehearsal.

After the flight, stop host tracking, stop logging and inspect completion/error status. Save a final flight, reopen it paused and confirm expected telemetry, reference and local video. Retain the original session until the archive has been checked. Camera permissions, actual board I/O, pointer mechanics and the station network require equipment checks beyond the software test suite.

Symptom	Check
USB connected, no rocket data	Start Polling; verify board/port and radio. A port open is not a radio handshake. Inspect rejected packets and freshness.
Commands locked at Launch	Expected in LIVE until the software uplink-enable contract is implemented. DEMO exercises its labelled simulation.
Camera absent from selector	It may be selected, active or closing in another channel. Release it there, refresh, and check OS permission.
Rocket upright but unavailable	Neutral placeholder: gyro integrals do not establish attitude. Choose a valid reference explicitly for simulated pose.
No flame/canopy	Current telemetry has no confirmed fields; a reference may lack the required events. Do not infer them from phase.
Simulation has no usable path	Verify active saved motor/configuration, custom curves, chosen JAR and worker log; fewer than two valid rows is rejected.
Tracking stopped	Inspect freshness, GPS and connection; restart deliberately. A host stop cannot stop the last physical move.
Remote thumbnails empty	Station video transport is pending; source selection alone does not create a connection.
Controls crowded	Use the intended Launch display area; small-window layout improvements remain outstanding.

Part II.
Codebase and developer documentation

14. Repository, entry points and source ownership

The application is self-contained in the standalone `RT_Telemetry_UI` repository. Its `src/rocket_gnc_monitor` package is the Python application; `simulation_bridge/src/RocketBridge.java` is the Java integration layer. The old PyQt5 ground-station folder is a behavioral reference, not a runtime dependency. Production builds use bundled resources and do not need the original telemetry, OpenRocket source or font-system paths.

The console entry point `rocket-gnc-monitor-v0a` and `python -m rocket_gnc_monitor` both call `__main__.main`. The packaged launcher calls that same function. It parses arguments, registers Lucida Grande, names the Qt organization/application, resolves the per-user data directory, resolves opted-in startup choices and CLI overrides, and runs the controller-free wizard before constructing `StationWindow`. An explicit `StationProfile` fixes the role for this session. The window owns one `Controller`; Launch adds a companion Systems view over that same owner. Closing the main window invokes shutdown. Video roles gate serial callbacks as well as hidden actions.

```
RT_Telemetry_UI/
  src/rocket_gnc_monitor/    # application package
  simulation_bridge/src/    # Java bridge source
  resources/                # icon, fonts, demos, motors, examples
  docs/                     # in-app Markdown help and dependency notices
  tests/                    # unit, Qt, serial and engine integration tests
  tests/fixtures/           # captured legacy contract and small .ork fixture
  scripts/                  # setup, build and package verification
  packaging/                # PyInstaller spec, launcher, entitlements
  vendor/                   # staged engine, bridge, JRE, build manifests
  build/                    # generated jobs, logs and validation artifacts
  dist/                     # distributable app/DMG or folder/ZIP
  Makefile                  # shared native build entry points
  Makefile.macos / Makefile.windows
  pyproject.toml / uv.lock / requirements-build.txt
```

14.1. Module map

Paths link to the standalone release tag, so this report does not require the old application directories to exist in the parent repository. Use the named symbols rather than fixed line numbers as maintenance anchors.

Module	Responsibility and main seams
src/rocket_gnc_monitor/__main__.py	CLI/startup; Qt data path and fonts; startup, demo, flight and engine smoke modes.
src/rocket_gnc_monitor/controller.py	State owner and orchestration: connection generations, polling, command dispatch, tracking, mode changes, recording, replay, file jobs, alerts and shutdown.
src/rocket_gnc_monitor/ui.py	`MainWindow`, `MissionDialog`, workspace construction, shortcuts, file dialogs, result presentation, enabled states and plots.
src/rocket_gnc_monitor/rocket_panel.py	Complete legacy rocket Telemetry/Commands/Power/Recovery subviews and retained confirmations.
src/rocket_gnc_monitor/settings.py	Immutable `AppSettings`, validation, atomic per-machine persistence, runtime paths and JAR checks.
src/rocket_gnc_monitor/settings_ui.py	Modeless `SettingsDialog`, engine/folder pickers, defaults, validation and save/cancel.
src/rocket_gnc_monitor/widgets.py	Themes, `MountView`, animation and camera/pose transforms.
src/rocket_gnc_monitor/mount_geometry.py	Cached stand, turntable, beam, antennas and material definitions.
src/rocket_gnc_monitor/scene_mesh.py	Vectorized rigid transforms, normal shading and QPainter depth-sorted polygons.
src/rocket_gnc_monitor/flight_scene.py	Elapsed-time playback, quaternion SLERP, burn/recovery events and frame sampling.

Module	Responsibility and main seams
src/rocket_gnc_monitor/flight_view.py	Modeless 3D viewer, orbit/pan/zoom, timeline, monitor-time link and overlays.
src/rocket_gnc_monitor/rocket_geometry.py	Cached schematic rocket, ignition plume and recovery canopy.
src/rocket_gnc_monitor/fonts.py	Private font registration and global Qt font choice.
src/rocket_gnc_monitor/domain.py	`Mission`, `Sample`, finite checks, geographic transforms, pointing and wind validation. Contains a synthetic test-data helper.
src/rocket_gnc_monitor/location.py	Qt-independent coordinate conversion and `Location`.
src/rocket_gnc_monitor/location_ui.py	`LaunchLocation` widget; format/preset state, preview and edit-versus-display distinction.
src/rocket_gnc_monitor/devices.py	Port identity inventory and bounded independent `SerialWorker` threads.
src/rocket_gnc_monitor/protocol.py	Incremental `ZephyrusDecoder`, pointer packet builder and legacy route-validation helper.
src/rocket_gnc_monitor/zephyrus.py	Rocket command opcodes/packet building and canonical 43-column legacy CSV adapter.
src/rocket_gnc_monitor/legacy_sample.py	Restricted AST-based CSV literal parsing and full row-to-sample conversion.
src/rocket_gnc_monitor/legacy.py	Validated legacy CSV-to-session conversion, provenance and incomplete-import handling.
src/rocket_gnc_monitor/demo.py	Bundled CSV integrity, launch cue, independent receiver clocks and plot-only ENU frame.
src/rocket_gnc_monitor/recording.py	Asynchronous `SessionRecorder`, indexed read-only `SessionReader`, raw capture CRC reader.
src/rocket_gnc_monitor/flight.py	Portable archive snapshot, atomic save, validated extraction and `LoadedFlight`.
src/rocket_gnc_monitor/media.py	FFmpeg location, camera discovery, process arguments, frame queue, recording and orderly stop.
src/rocket_gnc_monitor/virtual_pointer.py	Port-free legacy packet simulator, bounded slew model and independent georeferenced `VirtualFlight` playback.
src/rocket_gnc_monitor/trajectory.py	`Trajectory`, online wind normalization, motor resources and isolated `SimulationJob`.
src/rocket_gnc_monitor/station_profile.py	Immutable station/role/vehicle/site profile, channel/target assignments and opt-in startup persistence.
src/rocket_gnc_monitor/startup_wizard.py	Four-page setup without a device controller; acceptance and remember preference.
src/rocket_gnc_monitor/station_workspace.py	StationWindow/StationDisplay, two Launch displays and reduced Away/Video layouts sharing existing instruments.
src/rocket_gnc_monitor/site_presets.py	URRG launch and numbered-station coordinates, default mission initialization.
src/rocket_gnc_monitor/rocket_state_badge.py	Received phase color, source mode and stale-state annotation.
src/rocket_gnc_monitor/rocket_pose.py	Telemetry orientation/status validation, provenance and reference-frame conversion.
src/rocket_gnc_monitor/rocket_pose_view.py	Large compact rocket with upright azimuth-only camera; no drag or scroll.
src/rocket_gnc_monitor/rocket_pose_panel.py	Source choice, raw gyro integrals, pose/effect status and labels.
src/rocket_gnc_monitor/video_wall.py	Launch primary channels, Away thumbnails, matching-channel promotion and freshness; no network transport.
src/rocket_gnc_monitor/iris_link_panel.py	Shared Iris stage target and DEMO-only switching placeholder.
src/rocket_gnc_monitor/launch_uplink_panel.py	One-board Launch uplink enable placeholder and simulated state.
src/rocket_gnc_monitor/launch_link_panel.py	Desired Away communication route and LTU topology/status placeholders.
src/rocket_gnc_monitor/station_io.py	Bounded receive-only KISS/APRS and one-peer read-only TCP snapshots.

Module	Responsibility and main seams
src/rocket_gnc_monitor/station_panel.py	APRS/LAN status, explicit connection controls and persisted endpoints.
src/rocket_gnc_monitor/wifi.py, src/rocket_gnc_monitor/wifi_panel.py	OS network discovery/settings and separate desired/current SSID presentation.
src/rocket_gnc_monitor/__init__.py	Package version identifier.
simulation_bridge/src/RocketBridge.java	Fork-specific OpenRocket loading, motor resolution, nominal options, trajectory/error publication and private preferences.

14.2. Dependencies and resource resolution

Python is the main application language because Qt, serial I/O, numerical plots and filesystem orchestration fit one runtime. Java is isolated to the existing team OpenRocket fork. The GUI uses PySide6 Essentials and PyQtGraph with NumPy; serial transport uses pyserial, video uses imageio-ffmpeg's native FFmpeg, and location conversion uses `mgrs` and `openlocationcode`.

The project permits Python 3.12 through 3.14 in metadata; the build guidance and tested environments center on 3.12–3.13. Dependency ranges live in `pyproject.toml`; `uv.lock` and `requirements-build.txt` pin build environments. Do not treat the range bounds as a tested matrix.

Source execution resolves resources from the repository root. Frozen execution uses PyInstaller's `sys._MEIPASS`. FFmpeg is located through its wrapper; Java/engine/bridge are resolved under `vendor`. A wheel or editable Python package alone is not the complete production distribution: package the resource tree and native runtimes through the supplied build workflow.

15. Runtime architecture and state ownership

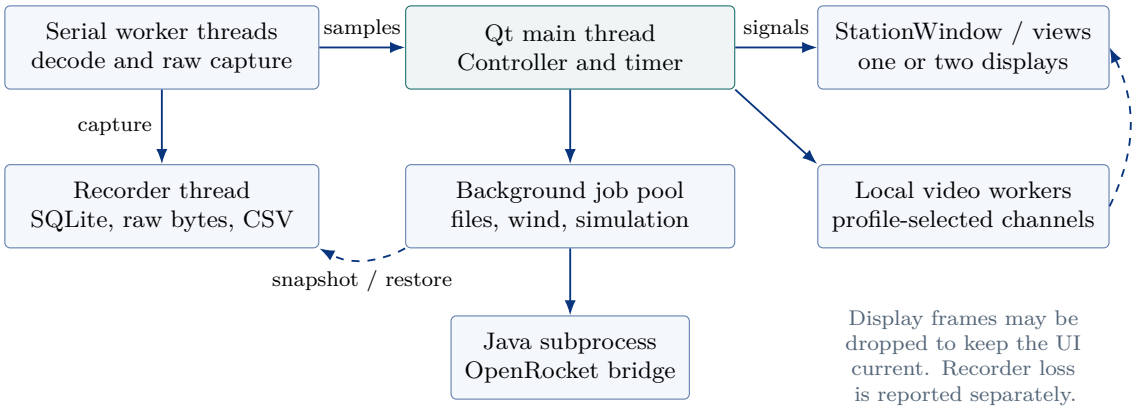


Figure 8: Runtime ownership. Hardware reads, file compression and engine execution do not belong in Qt paint or input callbacks. LIVE accepted samples enter the recorder before the display queue.

15.1. Concurrency model

Qt widgets and controller state are updated on the main thread. Serial workers own their ports and decoders. The recorder has its own thread and SQLite connection. General background jobs use a four-worker executor; video open/close operations use one single-worker executor per stream to serialize its FFmpeg lifecycle changes independently. Each decoder thread drains its FFmpeg frames, with a separate stderr drain.

The controller's 30 ms timer drains up to 2,000 queued messages per tick, advances DEMO/REPLAY, dispatches tracking when due, takes the most recent frame from each video stream, collects completed futures and updates alerts. Playback delta is capped at 0.3 s per tick, so a long blocked GUI does not produce an arbitrarily large playback jump. This desktop scheduler is not a hard real-time control loop.

Buffer / cadence	Bound	Consequence
Controller messages	16,000 messages	UI overflow increments `ui_drops`; stale generations are ignored.
Displayed history and track	6,000 each	Bounded UI memory; not the full session archive.
Plot time-series tail	1,200 samples	Altitude/rate/rotation plots use a shorter recent tail.
UI refresh	About 10 Hz	`refresh` throttles updates to 0.1 s. Some tables refresh every 0.5 s.
Telemetry command queue	64 commands	Bounded; each command expires after 0.5 s.
Pointer command queue	1 command	No silent overwrite; a pending dispatch prevents another ordinary target.
Tracking	At most 5 Hz	New target only if the preceding one is not pending.
Recorder queue	16,000 entries	LIVE overflow records loss/fault. File import can wait rather than drop rows.
Recorder commit	About every 0.25 s	Raw/CSV are flushed and fsynced before committing matching indexes/state.
Video display queue	2 frames per stream	Old display frames are dropped for freshness, independently of FFmpeg recording.
Replay history reconstruction	Previous 120 s	Earlier samples remain on disk and can be sought explicitly.

In LIVE, `Controller.enqueue` sends an accepted sample to the recorder before putting it into the UI queue. Raw callbacks also reach the recorder directly. Thus a dropped UI message need not mean the same sample was lost from disk. Conversely, recorder overflow is an independent data-loss condition. Do not merge these counters or move capture entirely behind UI repainting.

15.2. Connection generations and asynchronous results

A new serial connection receives a generation token. Worker messages are used only if the current role still owns that generation and the mode is LIVE. This prevents a late error/sample from an old worker from altering a new link. Each video stream uses a separate generation and lifecycle executor to prevent an old open result from replacing a newer selection. Camera reservations cover pending opens and closes as well as running capture.

General jobs report through the controller's future list and `task_done` signal. Weather/reference/simulation jobs retain source-mode identity; completed work from a different mode is not silently installed. Simulation results are also compared against the relevant mission inputs. The UI performs additional weather site checks and reference-origin validation.

Flight operations use `flight_busy`. Saving captures copies of mutable mission/reference/history state before background I/O. Opening stores the connection generation and mission snapshot; if source/settings change while loading, the result is not applied over the newer context.

15.3. Mode transitions

`switch_mode` validates the target and verifies a requested demo resource before changing the current source. It stops logging, closes replay/serial/video resources, cancels host tracking, clears histories and command state, resets time and sample counters, clears pointer calibration and installs the new mode. DEMO saves the prior reference for restoration when leaving.

Figure 3 shows the shared source-transition boundary.

`open_replay` validates the session's mission/reference before replacing the current source, then enters REPLAY, restores the manifest, seeks to zero and stays paused. `apply_flight` restores an extracted flight's mission, reference and saved position; a planning-only file forces a clean LIVE state. Neither path restores a physical connection.

15.4. Lifecycle and shutdown

`Controller.shutdown` is idempotent. It stops the Qt timer, stops serial workers, closes video and waits for all local video executors, cancels the active simulation, closes recorder/reader resources and waits for background work. Pending recording-close work is allowed to finish. Keep this ownership order when introducing a new worker so it cannot write to an already closed resource.

16. Station profiles, routing and future transport

16.1. Startup persistence and role boundaries

`StationProfile` validates station, Away number, role, vehicle and launch site, and exposes local/system channels and telemetry targets. The historical internal station key `base` remains for compatibility but displays as Launch station. Public `launch` normalizes to that key. Normal Launch construction passes the single-board `launch` controller layout; `base/legacy` layouts remain compatibility APIs.

Opt-in choices live in `startup-choices.json`; legacy `station-profile.json` and `station-layout.json` do not seed startup defaults. Writes are validated/atomic. `__main__.py` applies remembered choices then explicit CLI overrides, waits for wizard acceptance, and only then creates a workspace. The wizard has no controller or transport side effects. Cancelled setup must not alter remembering preferences or construct workers.

`station_workspace.py` assembles role cards over one inherited instrument owner. Explicit profiles cannot switch identity mid-session. Existing legacy construction still supports its programmatic layout switch. Video role rejects serial connection callbacks even if invoked through hidden actions; recording and file/replay remain usable. Both Launch windows update from one controller. Closing the companion view must not accidentally destroy shared instruments.

16.2. One Launch board, Away routes and placeholder state

Launch has one telemetry worker for downlink and future uplink. Away has downlink and pointer workers. Controller authorization combines board layout, source mode and actual supported protocol. Launch LIVE uplink-enable remains unimplemented; no command may be enabled by a route dropdown or remote data. Iris has one shared target on the Launch board, with no independent uplink target or second serial role. DEMO records simulated target/enable events and clears them when leaving that mode.

Remote pointer selection is a desired route, not a transport or measured pose. Keep desired, dispatched and confirmed states distinct when firmware/network adapters are added. A port opening is not evidence of rocket communication or board switch position. Preserve generation checks and uncertain-write behavior.

16.3. Video-wall integration contract

The controller owns an immutable tuple of local channel IDs. It rejects unknown or inactive streams; each has a separate lifecycle, reservation and recorder. Launch Video uses local `digital`; Balius Away uses `digital,analog`; Iris Away 1–3 use those same IDs with Sustainer labels; Away 4 uses `digital,analog2`. System channels for Iris include all three.

`StationProfile.away_channels` validates each remote station/channel pair. On the GUI thread, a future transport may call:

```
wall.update_remote_frame(
    "away4", "analog2", pixmap,
    quality="transport-provided quality", received=local_monotonic_time)
wall.select_remote("away4", "analog2")
wall.select_local_digital()
```

Use `mark_remote_unavailable` on link loss. Receive time must be local monotonic time; it is not an unsynchronized sender timestamp. Five seconds without fresh frames becomes stale. Quality text comes from the transport and is not guessed from image appearance. `set_local_source` separates live camera from paused/file/demo context; `set_local_frame` does not overwrite a promoted remote view. Source promotion replaces only a matching channel.

No network worker, relay, remote recorder or remote-replay path is implemented by the wall. Define those lifecycles, timing and provenance before connecting an actual station transport. Existing archive support preserves arbitrary local channel directories without changing schema; filtering inactive channels for a role never deletes archive assets.

16.4. Existing networking and the fusion boundary

`station_io.py` decodes bounded KISS frames and supported uncompressed AX.25 APRS positions. Its explicit `StationLink` is a separate one-peer read-only snapshot service. Validate lengths, schema and finite values before presenting data; peer values remain outside accepted local samples and hardware command readiness. `station_panel.py` persists endpoints; Wi-Fi helpers read current/saved SSIDs and open OS settings without storing credentials.

There is no sensor-fusion estimator in v0a. Multiple GS demo logs are independent recordings; remote summaries and APRS do not merge into an attitude or navigation state. A future fusion module needs vehicle/source IDs, duplicate packet handling, time alignment and clock uncertainty, coordinate/altitude datums, measurement covariance, freshness/reboot rules and estimator validity. It should produce a separately identified estimate with provenance and uncertainty while preserving raw inputs. An estimator output must not acquire command authority merely by appearing in a display. This is an extension boundary, not an implemented algorithm or claimed navigation accuracy.

17. Rocket orientation and flight-effect contracts

`rocket_pose.py` separates raw telemetry integrals, validated orientation and reference frames. The rocket nose is body $+Z$; the world frame is ENU. The current Zephyrus adapter supplies no orientation or verified flight-effect object. A future decoded sample can provide:

```
sample.details["orientation"] = {
    "frame": "body_to_ENU",
    "body_axis": "+Z",
    "quaternion_wxyz": [w, x, y, z],
}
sample.details["flight_status"] = {
    "motor_ignited": True,
    "motor_burning": False,
    "parachute_deployed": False,
}
```

These are internal adapter contracts, not new serial fields or firmware commands. Quaternion components must be finite, scalar-first and within 0.001 of unit norm; accepted rounding error is normalized. Wrong frame/axis, invalid values or absent orientation leave the neutral unavailable pose. The renderer never derives orientation from phase, climb or a nominal path.

Flight-status values must be exact Booleans. Missing/malformed values remain unknown; historical ignition alone does not establish burning. Phase and pyro bits cannot establish recovery. Simulation frames use their own validated quaternion/event path and never backfill telemetry. Missing or partial event history retains unknown effects or an explicitly illustrative pose.

`rocket_pose_view.py` caches geometry, uses an upright azimuth-only camera and fits the body plus any displayed effects at a stable scale. `rocket_pose_panel.py` owns the source selector and numeric/status labels. Keep camera rotation separate from body orientation and control dispatch. Full trajectory and antenna views retain orbit interaction; applying their mouse controls to this compact pane would change the intended interaction.

18. Data contracts, units and geographic frames

18.1. Mission and Sample

`Mission` is a dataclass of persisted configuration. `validate` checks coordinate/numeric ranges, supported location/height choices, actuator count, seed/index, motor-path list and wind profile. `save` writes schema version 1 through temporary-file replacement. `load` accepts version 1, filters to known fields, uses dataclass defaults for absent

fields and always clears `pointer_calibrated`. There is no general multi-version migration framework. Station presets additionally store `pointer_site_name`; see `site_presets.py` for the URRG pad and station catalog.

`Sample` is a dataclass representing one accepted or imported sample. It contains device/receive times, source, packet number, navigation/kinematic values, phase, actuator dictionary and extensible details. Optional missing measurements use `None`; they are not replaced by zero. The live decoder converts nonfinite floating sensor values to unavailable values. JSON output rejects NaN. Appendices A and B give all fields.

`details` carries original legacy fields, provenance, boot index and quality annotations. Extend it conservatively: legacy rendering, CSV export, record/replay and archive validation all consume samples. New actuator entries should use explicit `demand`, `measured` and/or `drive` keys with declared units, rather than renaming the four unknown legacy channels.

18.2. WGS84 and local ENU

`ecef` uses WGS84 semi-major axis $a = 6378137$ m and eccentricity squared $e^2 = 6.69437999014 \times 10^{-3}$. For latitude ϕ, longitude λ, ellipsoid height h:

$$N(\phi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}}, \quad \boldsymbol{x} = \begin{bmatrix} (N + h) \cos\phi \cos\lambda \\ (N + h) \cos\phi \sin\lambda \\ (N(1 - e^2) + h) \sin\phi \end{bmatrix}.$$

`to_enu` subtracts the origin ECEF vector and applies the standard local east/north/up rotation at that origin. The plotting contract is (E, N, U), with $E \times N = U$. `pointing` converts the local displacement to

$$\text{az} = \text{atan2}(E, N) \bmod 360^\circ, \qquad \text{el} = \text{atan2}\big(U, \sqrt{E^2 + N^2}\big).$$

The geographic helper rejects degenerate directions; virtual rehearsal handles exact zenith by retaining the previous azimuth, as described in Part I. The live legacy pointer caller deliberately chooses its own origin/height convention; it does not call the mission trajectory conversion as a substitute.

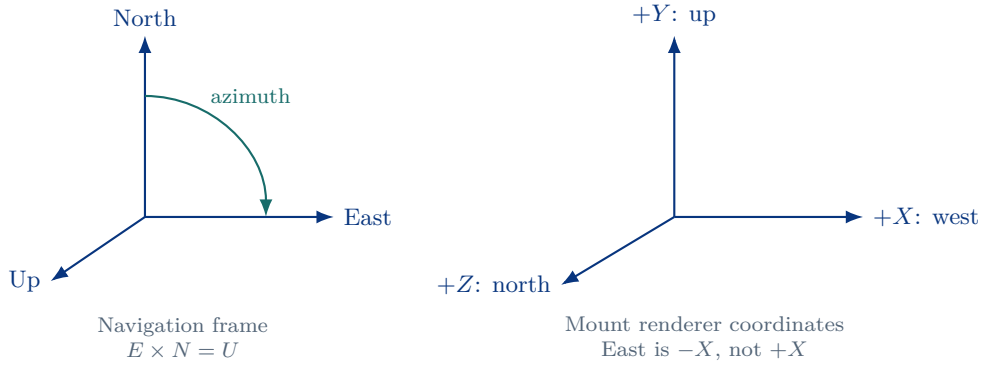


Figure 9: The navigation and renderer bases are different but both preserve the intended chirality. The diagram is a projected axis sketch; up on the page is not a universal physical direction.

18.3. Time normalization and reboot behavior

The decoder extends a 32-bit millisecond rollover when a backward jump exceeds 2^{31}. A smaller backward jump starts another boot epoch, resets rollover offset and clears sequence-gap history. Sequence gaps use modulo-65536 differences and count missing values only for forward deltas between 2 and 32767. The controller uses boot-index changes to invalidate alignment and old history.

For recording, elapsed time is the host monotonic receive time minus recorder start, rounded to microseconds. Replay is driven by this elapsed column, not by pretending onboard uptime began at zero. Events restore alignment and last sent pointer state. Imported CSV uses original receive timestamps relative to its first row. DEMO uses its receiver's own timestamps; it does not merge the three clocks.

19. Serial and protocol implementation

19.1. Port ownership and transport rules

`ports` returns device, description, serial number and an identity derived from VID/PID/serial when possible. `Controller.connect` enforces LIVE, nonempty selection and distinct board ports, then constructs the worker. Connecting does not start telemetry polling or transmit a setup command.

The worker opens pyserial with the role-specific read timeout and a 0.2 s write timeout. Queued writes call `cancel_read` where available to wake the worker. Only the telemetry role has the polling gate. Pointer RX is logged as bytes and hex status, not decoded as authoritative measured angles. Each write checks expiration, records the actually written prefix, and emits `sent` only if the complete payload was written. Errors close the port; there is no retry loop.

19.2. Telemetry framing and checksum

A telemetry frame is 144 bytes:

```
offset 0..1      AB AB synchronizer
offset 2..129    128-byte Zephyrus payload P
offset 130..143 14-byte ground receiver trailer G

P[127] = sum(P[0:127]) modulo 256
```

The incremental parser retains partial frames and a trailing potential sync byte. After a bad checksum it advances one byte and rescans, so an overlapping valid synchronizer is not thrown away. It accepts concatenated reads and reports accepted, rejected, discarded-byte and inferred-gap counters. The additive checksum covers the payload only, is weak error detection and does not authenticate a sender.

Payload offsets are relative to $P[0]$; multibyte fields below are little endian unless stated otherwise.

Bytes	Field	Decode / interpretation
0–1	Pyro continuity	Six two-bit states from the packed 16-bit value.
2, 3	Armed, fired	Channel bit masks; channels 0–5.
4–9	Pyro resistance	Six unsigned bytes divided by 10.
10–15	Servo drives	Four packed 12-bit values, retained as legacy drives.
16–24	Acceleration XYZ	Three signed 24-bit integers divided by 12800 and multiplied by 9.80665; m/s^2.
25–30	Gyro XYZ	Signed int16 values times 0.03051757812; legacy Y sign inverted; degrees/s.
31	Rocket GPS fix	Raw fix code; 3D trajectory requires at least 3.
32–39	Latitude, longitude	Two signed int32 values times 10^{-7} degrees. Invalid bounds clear coordinates/fix.
40–43	GPS height	Float32 metres; interpretation is mission-dependent.
44–51	Horizontal/vertical accuracy	Two uint32 values divided by 1000.
52	Satellites	Unsigned byte.
53–58	Raw pressure, temperature	Two 24-bit raw words; temperature calibration is in the legacy adapter.
59–62	Filtered altitude	Float32 metres.
63	State	0 ground testing, 1 preflight, 2 flight, 3 post-apogee, 4 main, 5 end.
64–75	Integrated rotations	Three float32 values at 64, 68, 72; raw roll/pitch/yaw gyro integrals, not Euler attitude.
76–79	Maximum heights	Barometer then GPS uint16.
80–83	Onboard clock	uint32 milliseconds uptime.
84–85	Packet sequence	uint16 modulo counter.
86	Onboard RSSI	Signed byte minus 99.
87–92	Cell voltages	Three signed int16 values divided by 1000.
93–94	Total current	Signed int16 divided by −1000.
95–97	BMS	Temperature byte divided by 2; protection status and enabled bits.

Bytes	Field	Decode / interpretation
98–109	Six rail voltages	Signed int16 times 0.0016.
110–121	Six rail currents	Signed int16 times 0.000625.
122–125	Velocity	Float32 metres/s.
126	Unmapped byte	Not interpreted by the present decoder.
127	Payload checksum	Sum modulo 256 of bytes 0–126.

Trailer G is RSSI at 0 (signed byte minus 99), fix at 1, signed int32 latitude at 2, longitude at 6 (both 10^{-7} degrees), and signed int32 height at 10. The legacy adapter divides receiver height by 1000 even though the audited firmware supplies metres. Preserve this discrepancy explicitly; the legacy pointer origin instead uses height zero.

Rotation fields Firmware accumulates roll from $-\omega_x\,dt$, pitch from $\omega_y\,dt$ and yaw from $\omega_z\,dt$, resetting integrals on entry to Preflight and Flight. These are independent body-axis totals. The historical `Sample.attitude` name does not make them a world attitude; current rendering never composes them as Euler angles.

19.3. Pointer command packets

```
byte 0       AA
byte 1       opcode
bytes 2..5   azimuth float32 LE (opcode 0), otherwise zero
bytes 6..9   elevation float32 LE (opcode 0), otherwise zero
byte 10      sum(bytes 1..9) modulo 256
```

Opcodes are 0 absolute, 1 UP, 2 DOWN, 3 LEFT, 4 RIGHT and 5 ZERO. The legacy firmware negates internal azimuth and uses a shortest-route policy; the host preserves original direction opcodes rather than reinterpreting them. Absolute finite values are serialized without invoking the old `validate_route` helper.

`validate_route` and mission envelope/calibration fields remain in source for compatibility and tests of that helper. They are not called by the current manual control/tracking dispatch path. Do not document them as an active mechanical interlock or silently enable them when modifying the legacy UI.

19.4. Rocket uplink packets

```
byte 0       AA
bytes 1..12  zero/reserved, except:
             byte 12 integer argument, or
             bytes 9..12 float32 LE angle
byte 13      command opcode
bytes 14..15 sum(bytes 1..13) modulo 65536, BIG endian
```

The mixed endian checksum is deliberate and covered by captured vectors.

Opcode	Encoder command	Argument
01 / 06 / 07	Arm / fire / disarm pyros	Nonempty bitmask of channels 0–5.
02	Advance state	Requested state code; UI uses current + 1.
03 / 05	Airbrakes / roll servo	Finite float32 angle.
04	PD activate	No parameter.
08 / 09	Zero servos / P-Y-roll	No parameter; zero-roll API aliases 09.
0A / 0B	Zero altitude / velocity	No parameter.
10 / 11 / 12 / 13	Piston / BP wells / tender descender / all	No parameter.
14	VTX power	Index 0–3 for 1/3/5/8 W.
15	Converter requests	Six ordered Boolean request bits.
16	BMS protections	Boolean encoded 0/1; API exists, operator status is read-only.

`rocket_packet` validates command/value types. UI confirmation and transport authorization remain separate layers: construct bytes without side effects, confirm the original sensitive actions in `RocketPanel`, then dispatch through the controller. Normal Launch LIVE authorization is blocked pending the uplink-enable protocol (and Iris

target contract). Retained encoders and legacy APIs do not imply that those commands are authorized in every profile.

20. Recording, replay and portable archive internals

20.1. Session writer and durability

`SessionRecorder` creates `session_YYYYMMDD_HHMMSS_<suffix>` and writes an initial manifest with `complete:false`. It enqueues sample dictionaries, events and role-tagged raw byte chunks. One writer thread owns all session output. SQLite uses WAL and `synchronous=FULL`; ordinary commits occur about every 0.25 s and at closure.

```
CREATE TABLE samples(
  id INTEGER PRIMARY KEY, elapsed REAL, t REAL, data TEXT);
CREATE INDEX sample_time ON samples(elapsed);
CREATE TABLE events(
  id INTEGER PRIMARY KEY, elapsed REAL, data TEXT);
CREATE INDEX event_time ON events(elapsed);
CREATE TABLE raw_index(
  id INTEGER PRIMARY KEY, elapsed REAL, role TEXT,
  offset INTEGER, length INTEGER);
CREATE TABLE snapshot_state(
  id INTEGER PRIMARY KEY CHECK(id=1), data TEXT);
```

`samples.data` is JSON from `Sample.to_dict`; `events.data` is `{"name":..., "data":...}`. A raw index offset points to the chunk header in `raw.bin`; its length is the payload length, not the whole chunk length.

Before a database commit, raw bytes and both CSV outputs are flushed and fsynced. The same SQLite transaction stores a `snapshot_state` row with matching raw/CSV byte lengths, manifest snapshot, duration, error and dropped count. A successful final close checkpoints WAL and updates the manifest with completion, duration and error/loss information. Queue overflow marks loss; a close timeout or writer exception leaves an incomplete/error condition.

The recorder's own decoder watches raw telemetry RX to produce the rejected packet CSV. Bad packets do not become `samples` rows. Good and bad CSV field order comes from `CSV_FIELDS`; the adapter retains the original padding and calibrations while fixing the documented old bad-packet-only inconsistencies.

20.2. Raw capture format

Each raw chunk has a 25-byte little-endian header with `struct.Struct("<4sddBI")`:

Offset	Type	Meaning
0–3	4 bytes	Magic `RGM1`.
4–11	float64	Host session elapsed seconds.
12–19	float64	UTC Unix seconds.
20	uint8	Role: telemetry RX 1, pointer RX 2, pointer TX 3, telemetry/rocket uplink TX 4.
21–24	uint32	Payload byte count.
25 onward	Byte payload	Exactly the declared number of raw serial bytes.
Following payload	uint32 LE	CRC32 of header plus payload.

`read_raw` yields complete CRC-checked chunks and rejects wrong magic, payloads over 4 MiB, truncated headers/payloads and CRC mismatch. This capture CRC protects stored chunks; it does not add authentication to the radio or repair the unchecksummed receiver trailer.

20.3. Replay reader

`SessionReader` opens SQLite read-only, reads manifest version 1, gets sample count and maximum sample elapsed time, and runs `PRAGMA quick_check`. `at(t)` returns the latest sample at or before t; `between(a,b)` returns $a < elapsed \le b$, ordered by elapsed and id, with a default 20,000-row limit. `events(t,begin)` uses the same temporal selection convention.

`seek` clamps to the session duration, clears future state, resets initial time alignment, applies prior events and reconstructs a 120 s history window. Received monotonic times are refreshed for display. Normal play advances the cursor at the selected speed, applies events and samples in the traversed interval, and pauses at the end. Events currently restore flight-zero and sent-pointer state; do not assume every console string is executable replay logic.

20.4. Flight archive layout

The portable extension is `.rktflight`; its contents are ZIP64 with a `flight.json` inventory. Files normally use DEFLATE level 3, while MKV is stored without another compression layer.

```
flight.json                 # format/version, source/scope, position, inventory
mission.json                # version 1; asset paths relative to this archive
assets/rocket.ork            # optional selected model
assets/motor_000.eng         # optional selected curves (.eng or .rse)
reference.csv
reference.json              # optional current normalized reference
session/manifest.json       # only if a session was exported
session/session.sqlite      # standalone staged database, no -wal/-shm
session/raw.bin
session/telemetry.csv
session/telemetry_badpackets.csv
session/legacy-source.csv   # if available from legacy import
session/reference.*         # if present
session/video*/segments.csv
session/video*/segment_*.mkv
```

The inventory records byte count and SHA-256 for every payload file; it does not recursively checksum itself. Top-level keys include `format:"rocket-gnc-flight"`, `schema_version:1`, app version, save UTC, source mode, export scope, sample count, optional session path, playback position, flight-zero/alignment and video-scope description.

Selected user motor files are embedded under generated portable names. The app's own standard/bundled motor library is part of the runtime distribution; it is not duplicated in every flight. Original job hashes and request metadata may remain in the reference manifest as provenance; the live model/motor paths are rebound to extracted assets.

20.5. Active snapshot consistency

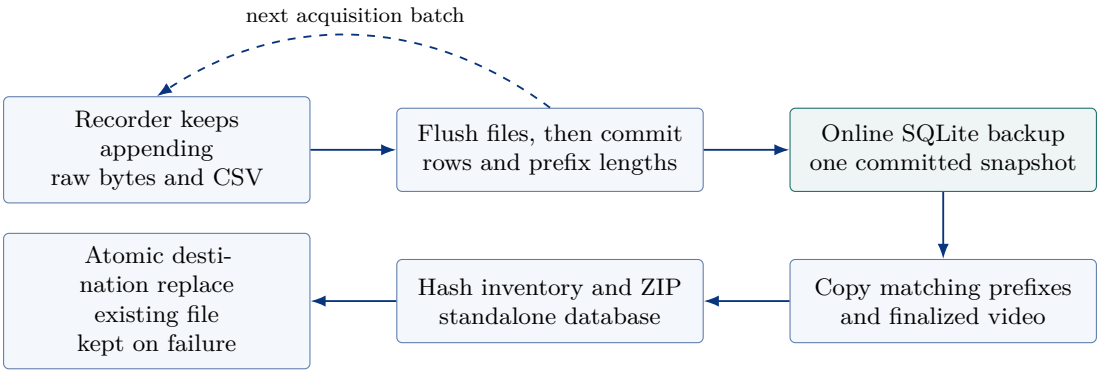


Figure 10: Save during logging. The snapshot boundary is a committed database transaction with matching file lengths, not an uncontrolled copy of an active SQLite file.

`snapshot_session` uses SQLite's online backup API to capture a committed database without pausing the serial writer. It then reads the copied `snapshot_state` to find exactly which raw/CSV prefixes belong to that database version. Later appends to the source do not change those prefixes. An early save waits briefly for the first committed state. Older stable sessions without snapshot state use their available files and retain their completion status.

Video is copied from finalized entries in the segment CSV; a partial trailing index line is ignored for active capture, and segments beyond the snapshot duration are omitted. After logging stops, the controller waits for pending video/recorder closure before saving. No unfinished camera file is claimed as complete footage.

The staged database is checkpointed and changed to DELETE journal mode once all staging readers close. Only this owned staging copy is changed; the active source database stays in WAL mode. This avoids shipping

platform-specific shared-memory state. A temporary archive is fsynced and replaced atomically at the requested destination.

20.6. Archive validation and application

`load_flight` extracts into a new UUID directory under the owned flight cache. It rejects unsupported versions, encryption, linked/directory/duplicate case-folded entries, absolute/traversal/backslash/colon/control-character paths, mismatched inventories, declared sizes and content hashes. Loading limits are 64 GiB of expanded entries, 100,000 entries and 16 MiB per metadata object; free space is checked before extraction.

Mission asset paths must resolve to included assets. Mission/reference data, SQLite integrity, sample/event structure and video segment indexes are validated before the controller applies a `LoadedFlight`. On failure, the new extraction directory is removed. Parsing uses JSON and ordinary media/model formats, not pickle or evaluation of archive-supplied Python.

The archive inventory is corruption detection, not a signature or trust identity. A deliberately altered payload with a recomputed hash is still subject to schema checks, but is not proven to have come from the team. File limits and validation should remain aligned when evolving save and load behavior.

20.7. Read-only analysis example

Run analysis with the app's development environment so its domain contracts match the recording:

```
from pathlib import Path
from rocket_gnc_monitor.recording import SessionReader, read_raw

session = Path("/absolute/path/to/session_folder")
reader = SessionReader(session)
try:
    print(reader.count, reader.duration, reader.manifest["complete"])
    for elapsed, sample in reader.between(-1, min(10, reader.duration)):
        print(elapsed, sample.t, sample.altitude)
    reader.export_csv(session.parent / "analysis-export.csv")
finally:
    reader.close()

for elapsed, utc, role, payload in read_raw(session / "raw.bin"):
    print(elapsed, role, len(payload))
```

Do not feed replay events to a serial worker. Reading archive/session data is separate from authorizing hardware commands.

21. Legacy import and demo provenance

`numeric_list` uses a small AST visitor. It accepts numeric/Boolean literals, bounded list/tuple forms, unary signs and known one-argument NumPy numeric wrappers, plus known Boolean names. It rejects arbitrary calls and attributes; it never invokes `eval`. Numeric-array text is limited to 4,096 characters and arrays to 32 elements.

`import_legacy` rejects files over 200 MB and known bad-packet filenames, requires the audited core headers, checks monotonic receive timestamps and refuses empty datasets. File import can block on the recorder queue to preserve all rows. It stores source bytes and hash, records observed imported flight transitions, and explicitly marks raw packets unavailable. A failed import retains an incomplete/error manifest rather than presenting a completed session.

`DemoFlight` verifies the uncompressed SHA-256 and row count from `resources/demo/manifest.json`; gzip is lossless packaging. It computes receiver-relative times, locates the first FLIGHT row and exposes a launch cue and indexed samples. Each sample contains the original row/UTC, station, source file/hash and local-frame note. Plot-only rejection of invalid/far GPS does not erase the original values.

The functions `domain.demo_sample` and `Trajectory.demo` remain useful synthetic fixtures and examples. They do not drive the current operator DEMO telemetry. Avoid reconnecting these helpers to the source selector when adding features; the user-visible default is the actual GS2 log.

22. Video implementation and synchronization

`camera_devices` invokes bundled FFmpeg to enumerate AVFoundation or DirectShow devices (and Linux video devices in its existing branch). `VideoWorker.arguments` builds an argument list, without a shell, for camera, file or test-pattern input. It maps only video and disables audio.

The display pipe is `rgb24` at 960×540. The decoder reads exactly one full frame before queueing it; incomplete EOF frames are discarded. The two-frame queue trades display history for low latency. Recording is a separate FFmpeg output from the same input: MPEG-4 for live/test sources, stream-copy for local files, Matroska segments and `segments.csv`. The display scaling does not imply the stored camera stream has the same dimensions.

`Controller.video_streams` maps the immutable profile-selected local channel IDs (`digital`, `analog`, or `analog2`) to `VideoStream` state: worker, backend, configuration, generation, reserved cameras, replay key/pause state, lifecycle future and open error. `video_frame` carries the stream identifier and bytes; `video_reset` clears only that panel. The legacy `video`/`video_config` properties and `frame` signal remain first-stream aliases.

`start_video(..., stream="digital")` increments that stream's token and schedules work on its single-worker executor. Duplicate camera sources are rejected before altering either stream, including a camera whose prior handle is still closing. `stop_video(stream)` closes one feed; omitting the stream closes all active local feeds. The UI filters camera selections and uses `serial_device_key` to exclude active board ports and macOS aliases from the other serial dropdown. `stop` first writes `q` and continues draining output so FFmpeg can finish the trailer/index, then uses bounded terminate/kill fallback. Keep stderr draining independent so logs cannot block frame acquisition.

A `Video recording started` event records session elapsed time, source kind, video directory and `stream`. Every recording start allocates a unique `video_<stream>_<uuid>` directory, even when opens are queued rapidly. Stopping logging waits for all local stream lifecycle futures before closing the recorder, including a feed manually stopped just beforehand. Old events without a stream identifier map to Digital. Replay finds the eligible recording segment at

$$t_{\text{video}} = t_{\text{replay}} - t_{\text{start}} - \Delta_{\text{offset}}.$$

It chooses each stream's segment whose interval contains that time and seeks relative to its start. A manually stopped replay feed stays stopped until an explicit seek or playback restart. Playing checks segment transitions every 0.2 s; a replay key avoids reopening an unchanged segment. The reader's media events and segment indexes are cached so these checks do not repeatedly scan the event log. Gaps stop only the affected feed. Positive offset therefore delays video. Paused replay uses `single_frame`; playback uses the requested speed. This is host timing alignment, not camera exposure or hardware clock synchronization.

23. Weather and OpenRocket bridge contracts

23.1. Weather normalization

`weather_profile` parses timezone-aware ISO timestamps and normalizes to UTC. It caps response bytes at 4,000,000 and uses a 15 s network timeout. The nearest returned hour must lie within 3,600 s. It transforms geopotential height H to geometric height using $R = 6356766$ m:

$$h_{\text{AGL}} = \frac{RH}{R - H} - h_{\text{launch,MSL}}.$$

The returned object contains `layers`, `raw`, `source`, `request`, `endpoint` and `retrieved_utc`. Network work stays in the job pool. The mission stores that object in `weather_raw`. Manual profiles use exactly the same validated east/north layer representation.

23.2. Trajectory contract

A trajectory is an $N \times 4$ NumPy array (t, E, N, U) plus JSON metadata. It requires 2–200,000 rows, finite values and strictly increasing time. Loading caps the CSV at 40 MB. The manifest must declare version 1, ENU and `m,s`; a nonsynthetic reference needs a valid three-value geographic origin. `at(t)` interpolates each spatial coordinate linearly within coverage and returns unavailable outside it. Save produces CSV and a same-basename JSON. Optional motion columns are `qw,qx,qy,qz`, the three ENU velocity components in metres/s, and roll/pitch/yaw rates in radians/s. They form a matching $N \times 10$ array; missing values are blank/NaN, never infinity. Partial column sets are rejected. Metadata declares `visuals_schema_version=1`, `attitude_frame=body_to_ENU` and `body_axis=+Z`. A bounded `flight_events` list stores time, type, source name and stable source ID. The Java

bridge exports actual simulation samples/events; trajectory save and portable-flight snapshots preserve these additions without changing the base four-column position contract.

23.3. Simulation job boundary

The Python job copies the model and all explicit/bundled custom curves, writes `request.json`, launches the private Java executable and captures its output in `worker.log`. The request contains schema version, copied asset paths, bundled motor count, simulation index, launch latitude/longitude, MSL altitude, ellipsoid origin altitude, rail settings, seed, wind layers, output directory and nominal/inertia flags.

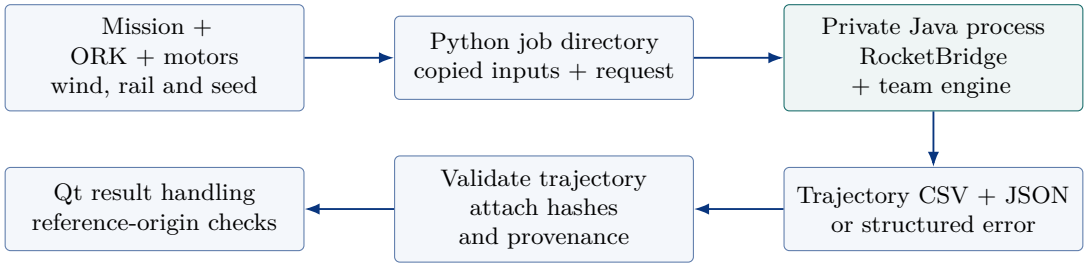


Figure 11: The simulation boundary uses files and a subprocess, keeping the JVM outside the Qt process.

Limits are 40 MB for the model file, 100 MB expanded ORK archive contents, 10 MB per motor curve, a configurable worker time limit (120 s by default) and 20 MB of worker log. Cancellation sets an event and terminates the child; timeout/oversized-log paths also stop it. Result collection checks return status, prefers a structured `error.json` message and otherwise includes a bounded log tail.

The bridge runs headless with private in-memory preferences. It loads the team's motor database, adds hash-verified bundled curves and explicit custom curves, loads the ORK document and verifies the selected saved configuration. A missing motor warning becomes a named error before simulation. A configuration without an active motor is rejected.

The fork-specific nominal setup disables saved simulation extensions and its global inertia override. It sets fixed rail direction rather than *launch into wind*, a 0.05 s step setting, a 600 s simulation maximum, ISA atmosphere, multilevel AGL wind and zero turbulence standard deviation. The Java wind direction is *toward*, $\operatorname{atan2}(v_E, v_N)$, in contrast to the operator's meteorological *from* input.

The first branch contributes time, X/east, Y/north and altitude/up. Nonfinite/nonincreasing rows are skipped; fewer than two rows is an error. Output metadata records branch count, motors and their engine digests, saved simulation name, warnings, nominal qualification flags and solver description. Python adds engine/model/motor hashes and the exact request/job directory. The bridge exits explicitly after success so engine loader threads cannot keep the worker alive.

23.4. Reproducibility and interpretation

The bundled MITRT N8406 curve is `resources/motors/TestLaunch_v14.eng`. Its engine digest is `057a820844b93fe3307401623a92d4c6`. Byte integrity and model-digest matching are separate checks: identical naming alone is insufficient.

A fixed seed and copied inputs support reproducibility on the pinned engine. Results do not automatically transfer to a different OpenRocket binary or qualified controlled-flight model. The package retains the complete team engine; a random upstream JAR is not a drop-in replacement for fork-specific fields. The exact source revision corresponding to the supplied engine binary remains a release provenance item in the existing build metadata.

24. UI, font and antenna graphics maintenance

24.1. UI responsibilities

Workspace builders in `MainWindow` construct widgets; event handlers call the controller or submit background jobs; `refresh` presents snapshots. `refresh_connections` and `RocketPanel.set_controls_enabled` express the existing board/mode rules. `guard` reports exceptions in the event area, status bar and warning dialog. Do not swallow a rejected operation by leaving a success-looking UI state.

`install_shortcuts` creates each QAction once. The original ten serial actions coexist with standard Open/Save flight actions. Binding signals again during periodic refresh would duplicate transmissions and must be avoided. The daylight toggle updates the shared palette, stylesheet and plot/widget colors. New custom-drawn labels should use the same font and palette conventions.

The global application font is Lucida Grande, registered from the packaged font collection by `configure_fonts`. The collection is loaded privately; the app does not install a system font. Plot axis labels and painter text use the same family. The report itself uses Latin Modern to match the requested LaTeX report style; these are separate presentation choices.

24.2. Where the antenna's physical description is defined

The physical description is procedural code in `src/rocket_gnc_monitor/mount_geometry.py`, specifically `mount_mesh`. It is compiled once into a cached mesh, then drawn by `MountView.paintEvent` in `src/rocket_gnc_monitor/widgets.py`. Local helper functions `box`, `rod` and `face` construct polygons; `palette` defines timber, frame, metal, grid, dark enclosure and other material colors. Faces are depth-sorted and projected with QPainter, so the drawing does not require an OpenGL/WebGL viewer.

The local transform functions deliberately separate attachment levels:

- `identity`: fixed stand/legs.
- `turn`: azimuth rotation for turntable and supporting structure.
- `carrier`: azimuth times elevation rotation, then translation to the pivot at renderer position $(0, 3.28, 0)$.
- `avenger`: carrier-local offset $(-1.03, 0, 0.16)$ for the enclosed Avenger assembly.

The common beam is centered at carrier-local zero with extents $2.65 \times 0.10 \times 0.10$ in illustrative model units. The Yagi boom is at local $x = 1.03$, running approximately from $z = -0.35$ to 3.55, with 21 crossed element positions. The grid is sampled procedurally; the feed extends along local $+Z$ from $z = 0.27$ to 1.04, in grid material. The dashed boresight continues on the same axis. These values define appearance, not fabrication dimensions.

`mount_rotations` and `MOUNT_COMPASS` implement the corrected basis: renderer $+Y$ up, $+Z$ north, $-X$ east. Do not flip only the compass letters to compensate for a reflection elsewhere. The regression checks $E \times N = U$, headings at the cardinal azimuths and determinant $+1$.

Camera yaw/pitch start at $72°/20°$. Dragging changes those at 0.45 degrees per pixel. Zoom is clamped to 0.65–2.5. The analytic right/up basis stays defined through top/bottom views and full orbit. Camera state and `set_pose` are separate; an orbit event must never call pointer dispatch.

24.3. Changing the model without changing hardware behavior

Edit only geometry/material/attachment transforms for visual changes. Keep attachment objects on `carrier` unless the physical part truly belongs to the fixed stand or azimuth-only support. Check front/side/top/perspective views, full orbit and zoom, feed/boresight collinearity, and the compass tests. Hardware behavior belongs in the controller/protocol layers and requires separate regression evidence.

25. Smooth animation and 3D flight rendering

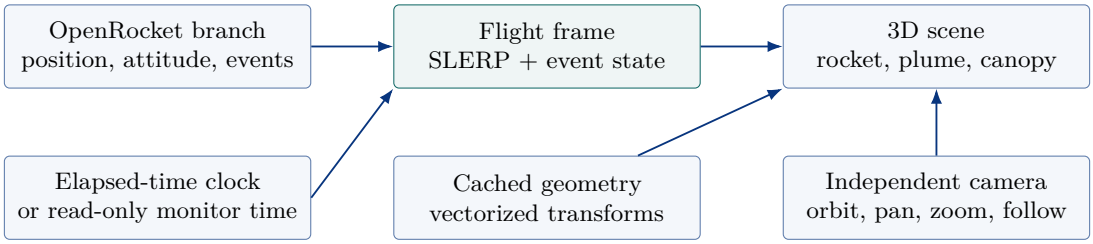


Figure 12: Flight events determine ignition and recovery appearance. Camera and display timing are independent of control dispatch; the antenna shares the cached-geometry renderer.

`Mesh` stores concatenated polygon vertices, group IDs, centers, normals and cached material shades. Array operations apply fixed/azimuth/beam transforms, project vertices and depth-sort faces; geometry is not rebuilt while painting. `MountView` uses a precise 16 ms Qt timer, stops it while hidden, samples virtual pose directly and approaches targets with a 45 ms exponential display time constant. Azimuth uses the shortest signed arc through north. Fixed camera framing removes pose-dependent zoom changes. Public command/pose state remains separate from interpolated display state and never generates serial commands.

`FlightScene` and `Playback` follow the team's OpenRocket v6.2 `TrajectoryData`, `TrajectoryPlayback` and `Trajectory3DPanel` conventions reviewed at commit `65defde5`. Playback uses a monotonic elapsed clock; orientation uses normalized shortest-arc quaternion SLERP. Samples whose angular rate times interval reaches π radians do not invent intermediate attitude. Missing orientation uses a labeled path-aligned illustration. Ignitions and burnouts pair by motor source; old burnout-only events begin at launch. Deployment enables the illustrative canopy; tumble/recovery marks engine-held attitude. No recovery or ignition is guessed from position.

`Flight3DDialog` has its own visible-only timer, snapshots reference identity, and resets when a reference changes. Monitor-time linking is read-only. Actual track overlays require matching mission/reference origins. The trajectory drawing uses at most 6,000 path vertices; full motion samples remain available for frame sampling. Cached schematic vehicle/plume/canopy geometry uses the same mesh renderer and Qt/NumPy runtimes as the antenna. No Python OpenGL dependency is added.

The 16 ms timer is an animation target, not a guaranteed frame rate. Package checks capture ignition, burnout and recovery from the bundled example and separately exercise the Zephyrus model. Native frame rate depends on window size, rendering workload and host hardware.

26. Virtual pointer transport and trajectory geometry

`VirtualPointer` satisfies the pointer worker's command interface without creating a serial object or worker thread. It accepts the same 11-byte packets, validates framing/checksum/opcode, queues one command and reports dispatch to the controller on the next tick. Absolute targets, four 5-degree jogs and ZERO update the simulated target. The model normalizes azimuth, bounds elevation to $[-90, 90]$ degrees and advances along the shortest azimuth route with bounded angular speeds. Virtual command reporting uses the separate `virtual_pointer_tx` label; command events include a `VIRTUAL` source.

`PointerLocation` uses the existing offline MGRS converter and keeps canonical absolute coordinates across display-format changes. Relative entry stores bearing b, horizontal distance d and altitude difference Δh. During mission validation, `pointer_from_launch` solves antenna latitude and longitude on WGS84 with $h_a = h_l + \Delta h$ so that the launch point in the antenna's ENU frame satisfies $E = d \sin b$ and $N = d \cos b$. A bounded Newton iteration uses metre-scaled finite differences and a 10^{-5} m residual criterion. Inputs are limited to 100 km; degenerate or nonconvergent coordinates raise a visible validation error. No additional runtime dependency is needed. Launch edits and disk loads re-resolve relative positions; absolute MGRS entry retains its canonical latitude/longitude. Old mission files default to lat/lon. Tests cover heading chirality, both hemispheres, high latitude, the antimeridian, zero distance, invalid entries and mission/flight persistence.

`VirtualFlight` owns its reference, independent clock, speed and antenna origin. For a launch-frame ENU point p_l, ECEF origins o_l, o_a and ECEF-to-ENU rotation matrices R_l, R_a, the antenna-frame vector is

$$p_a = R_a(o_l - o_a) + R_a R_l^{\mathsf{T}} p_l.$$

Azimuth is atan2(E, N) modulo 360 degrees; elevation is atan2$(U, \sqrt{E^2 + N^2})$. The trajectory's stored origin is authoritative, and both origins use ellipsoid height. Linear interpolation uses the existing trajectory contract. Coincident positions have no direction; exact zenith retains the preceding azimuth.

`Controller` snapshots reference identity and antenna coordinates for each rehearsal. A mismatch stops motion and clears the old rehearsal. The follow path requires an actual `VirtualPointer` instance and cannot dispatch through a physical worker. Manual actions cancel virtual playback; source changes close both connection types. Rehearsal never populates the telemetry history or track, and does not satisfy the ground-station readiness predicate.

`widgets.ComboBox` measures polished popup rows before opening, accounting for frame, padding and scrollbar space while bounding width to the screen. The stylesheet selects a list popup instead of the macOS menu-style checkmark delegate. This avoids the clipped mode/camera labels observed with Lucida Grande. The shared widget serves Settings, mission location and workspace selectors.

27. Settings storage and application

`AppSettings` is a frozen dataclass, independent of `Mission`. Schema version 1 contains `openrocket_jar` (empty means bundled), `sessions_directory` (empty means the per-user sessions directory), `daylight` (false by default), and `simulation_timeout` (120 s, validated integer range 10–1800). Nonempty stored paths must be absolute. The dialog expands user paths before storage. JAR validation checks the ZIP container for engine and dependency classes; API compatibility is established by bridge compilation and actual simulation, not by the file chooser alone.

`Controller` loads `settings.json` from its data directory. Malformed settings leave defaults active, preserve the original file and produce an operator-visible diagnostic. Missing external files remain selected until repaired; simulation does not silently substitute the bundled engine. Saving writes and flushes a temporary file in the same directory and then atomically replaces the settings file. In-memory preferences change only after a successful write.

`MainWindow` installs one application-wide Command-comma QAction with the native preferences menu role. `SettingsDialog` is modeless and reused while open. Save updates the controller and theme; Cancel has no persistence side effects. Every simulation receives the immutable settings snapshot at submission. The private Java runtime and bridge remain bundled; the first classpath entry is the selected engine. Result metadata records its path, SHA-256 and time limit. Logging resolves the configured destination when starting a new session. Portable-flight loading never rewrites machine settings.

28. Developer setup and native packaging

28.1. Prepare a standalone checkout

Use native 64-bit Python 3.12–3.13, JDK 17+ (`javac` and `jar`), and optionally GNU Make. Mac packaging also needs Xcode command-line tools. Windows requires x64 Python, including on an ARM host; verify `sysconfig.get_platform()` reports `win-amd64`. No import or build requires the former monorepo, old PyQt5 UI or an OpenRocket source checkout.

```
# Run from RT_Telemetry_UI, not the report folder.
make setup PYTHON=python3.13
make run
make demo STATION=launch
make test
make lint
```

Setup creates `.venv`, installs the pinned build requirements and the editable application. `uv sync -locked` is an alternative. Initial downloads require internet; offline builds need staged matching wheels and native vendor files. Metadata dependency ranges are not a tested interpreter matrix. The tested package/runtime provenance is recorded below.

28.2. Engine and private Java runtime

```
make runtime
make bridge OPENROCKET_DOWNLOAD=v6.2
# Alternative: a complete local team engine
make bridge OPENROCKET_JAR="/path/to/OpenRocket-MIT-v6.2.jar"
# Optional resolution at build time, not an app updater
make bridge OPENROCKET_DOWNLOAD=latest
```

Choose a local JAR or a download option, not both. The downloader queries `MDNich/ActiveControl_MIT_RktTeam`, selects the exact `OpenRocket-MIT-vX.Y.jar`, checks size and published SHA-256 when available, and records provenance in `vendor/engine.json`. If no published digest exists that fact is reported; a local hash is still recorded. Preparation stages a candidate and compiles before replacing an existing engine/bridge. Use a tested pinned release for production. A general upstream JAR may lack the required fork fields and motor resources.

Runtime preparation verifies OS/architecture and downloads a checksum-verified Temurin Java 17 runtime when needed. The bridge compiles for Java 17, records engine/bridge hashes and retains source/provenance. Rebuild it after editing `simulation_bridge/src/RocketBridge.java`. Never share `.venv` or `vendor/java` across OS/architecture targets. The runtime and engine are prepared at build time, not automatically downloaded by Settings or startup.

28.3. Build with or without tests

```
# On macOS, after setup: tests + package + packaged verification
make -f Makefile.macos all OPENROCKET_DOWNLOAD=v6.2
# On Windows with GNU Make, after setup:
make -f Makefile.windows all OPENROCKET_DOWNLOAD=v6.2

# Package only; no test/verification targets:
make package-macos OPENROCKET_DOWNLOAD=v6.2
make package-windows OPENROCKET_DOWNLOAD=v6.2
```

Each command runs on its target OS; PyInstaller is not a cross-compiler. An engine option stages the engine, bridge and matching runtime. Without an option, packaging reuses existing vendor files: prepare runtime/bridge first. The native `all` targets run test, package and verifier; run lint separately. Each checkout's default PyInstaller cache is `build/pyinstaller-cache`. An explicit `PYINSTALLER_CONFIG_DIR` overrides it; concurrent checkouts must not share that writable cache.

Windows PowerShell can use the scripts without Make:

```
py -3.13 scripts/build.py setup
.venv\Scripts\python.exe scripts/build.py runtime
.venv\Scripts\python.exe scripts/build.py bridge --download v6.2
.venv\Scripts\python.exe scripts/build.py package --platform Windows
# Optional validation, separate from the package-only command:
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m ruff check src tests --select F
.venv\Scripts\python.exe scripts/verify_package.py --model tests\fixtures\zephy_testlaunch.ork
```

Target	Outputs under ignored `dist/`
Mac ARM64	`RocketGNCMonitor-v0a.app`; `RocketGNCMonitor-v0a-Darwin-arm64.dmg`.
Mac Intel	Same native app name in its separate checkout; `RocketGNCMonitor-v0a-Darwin-x86_64.dmg`.
Windows x64	`RocketGNCMonitor-v0a/` and `RocketGNCMonitor-v0a-Windows-x64.zip`.
Universal Mac	`RocketGNCMonitor-v0a-universal.app`; `RocketGNCMonitor-v0a-macOS-universal.dmg`; ZIP wrapping the DMG.

Archives have SHA-256 sidecars. The bundle identity is `edu.mit.rocketteam.gncmonitor.v0a`, version 0.1.1. The migration preserves saved formats, application identity and separate v0a preferences. The PyInstaller spec bundles Python/Qt, native MGRS, fonts, FFmpeg, Java, engine/bridge, resources and notices, without unneeded WebEngine/QML or full scientific distributions.

28.4. Universal Mac composition

Build and verify ARM64 and x86_64 apps in distinct native environments; Intel can build/run through Rosetta on Apple Silicon. Then:

```
make package-macos-universal \
  MAC_ARM64_APP="/path/to/arm64/RocketGNCMonitor-v0a.app" \
  MAC_X86_64_APP="/path/to/intel/RocketGNCMonitor-v0a.app"
```

`scripts/package_macos_universal.py` verifies architectures, signatures, matching app versions/engines and Java provenance, preserves native inputs, copies symlinks, compiles a small universal launcher and signs nested payloads before the outer bundle. The launcher selects one complete native app and uses `execv`, preserving arguments and exit status. This is not a claim that every library is universal; concatenating finished PyInstaller archives with `lipo` is not the build method. `UNIVERSAL_OUTPUT` changes the destination. macOS 14+ reflects the included native runtime requirement.

28.5. Verification, signing and cleanup

Verify the extracted/relocated deliverable, including a path with spaces:

```
.venv/bin/python scripts/verify_package.py \
  --package "/path with spaces/RocketGNCMonitor-v0a.app" \
  --model tests/fixtures/zephy_testlaunch.ork \
  --output build/package-verification
codesign --verify --deep --strict "/path/to/RocketGNCMonitor-v0a.app"
```

Use the Windows interpreter and extracted directory on Windows. The verifier removes developer runtime overrides and uses minimal system PATH. Use native desktop Qt for Windows; offscreen Mac regression tests do not replace native visual checks. Universal verification must exercise both launcher slices.

Default Mac signing is ad-hoc. `MAC_SIGN_IDENTITY` selects an available Developer ID identity; notarization is separate. Windows is unsigned unless a separate signing workflow is applied. Preserve dependency licenses, Java legal files, engine/bridge provenance and the user-designated proprietary font notice. No signing credential belongs in source control.

Keep source, tests, small fixtures, authored documentation and provenance in Git. Keep environments, vendor downloads, recordings, captures and distribution binaries out. `make clean` in the *application* removes both build and dist, including finished packages. Remove only build when retaining final deliverables. The *report*'s separate Makefile clean removes its build intermediates while keeping the final PDF.

28.6. Manual Windows CI

`.github/workflows/build-windows.yml` provides a manually dispatched Windows Server 2025 build using x64 Python 3.13, JDK 17 and pinned requirements. It prepares v6.2, runs tests/lint, packages and verifies extracted output with native Qt/minimal PATH. Download the package/checksum artifact and verification artifact promptly: retention is one day. The workflow does not publish a release or run automatically. Recorded run metadata identifies the actual interpreter/JDK and source revision; hosted runners do not validate physical USB or radio equipment.

29. Testing and release evidence

29.1. Regression responsibilities

Area	Required coverage
Protocol / transport	Golden command vectors, fragmentation/corruption, wrap/reboot, timeouts, uncertain writes, independent ports and polling.
Parity / controls	All original CSV columns and shortcuts, genuine Qt key dispatch, confirmations, no duplicate signal connections, role/uplink gating.
Station setup	Wizard cancellation, remember/forget/CLI precedence, immutable identity, Launch/Away boards, URRG presets, two-view shared ownership.

Area	Required coverage
Video	Reservations through open/close, independent failure, station/channel assignments, promotion, stale source status, role isolation, local recording/replay.
Pose / graphics	Rejection of gyro integrals as attitude, quaternion/frame validation, unknown effects, reference provenance, upright camera, antenna chirality and interaction isolation.
Recording / flight	Queue/durability, active snapshots, finalized video, archive bounds/-path/hash rejection, atomic failure, replay without original assets.
Coordinates / simulation	Datum and relative-location signs, MGRS/Plus Codes, wind, motor resolution, real example and Zephyrus engine runs, motion/events and virtual pointing.
Network / packaging	Bounded receive-only data, no remote command authority, runtime architecture, download integrity, failure preservation and relocated packaged execution.

The captured legacy fixture records 118 rocket command cases across all 18 opcodes plus aliases, eight pointer cases, three tracking cases, 43 CSV fields and ten shortcut strings, with original source hashes. It was captured from the old class methods with physical connections replaced by memory streams. Do not regenerate the fixture simply to accept a protocol regression.

Original conventions retained include five-decimal receiver coordinates, zero receiver height for pointing, the receiver-height display division, servo scales/rounding and good-CSV padding. Explicit fixes include rejected packet counting, old bad-packet field inconsistencies, eight BMS columns and duplicate shortcut connections. These fixtures require no old source checkout.

29.2. 0.1.1 validation baseline

The following are completed migration/release checks from 25 October 2026, recorded in `docs/MIGRATION_VALIDATION.md`. Building this report does not rerun application/hardware qualification.

Check	Recorded result
Mac Apple Silicon	578 tests passed, no skips; regression suite used offscreen Qt.
Mac Intel / Rosetta	Full run: 573 passed, 5 skipped. Prepared-runtime simulation follow-up: all 5 simulation tests passed, covering four prior runtime skips. 577 Intel-applicable cases covered; the remaining ARM-host-only case passed on ARM.
Universal package	20/20 packaged checks for ARM64 and 20/20 for Intel under Rosetta. Fresh path with spaces and minimal PATH. DMG payload audit: 1,743 files/symlinks matched; deep/strict signatures passed.
Windows clean runner	567 tests passed, 11 platform skips; lint passed; 20/20 packaged checks. Windows Server 2025, Python 3.13.16 x64, Temurin 17.0.20.1+1.
Windows binary audit	ZIP checksum/CRC and embedded manifests matched; all 243 EXE/DLL/PYD binaries were AMD64.
Windows 11 ARM VM	Independent x64 source run: 567 passed, 11 skipped with Python 3.13.14 and v6.2. Adjusted virtual-pointer UI module: 7 passed.

Packaged checks cover wizard/roles, camera inventories, locked LIVE startup, all three demos, portable flights, URRG, virtual pointing, 3D ignition/recovery and bundled-example/Zephyrus simulation. The successful Windows run is Actions run 40213280209. Its preceding failure exposed a UI-test visibility/timing issue; the test now waits for visible/exposed controls before real mouse clicks, with behavior assertions unchanged. Production application modules were unchanged.

The delivered Mac was built from `8edfdb0`; Windows from `b61a153`. The release tag is `1923244`. Later commits contain workflow, test and documentation changes, with identical production application code. Both delivered archives use OpenRocket-MIT v6.2. A supplemental VM package using v6.4 ultimately passed 20 checks after an unexplained transient video-start failure and a separate guest-transport interruption; it is not the delivered ZIP.

Release artifact	Integrity
Universal Mac DMG	410,713,509 bytes. SHA-256: `acef899cf3b5eb2b43c402ce79c328df341f01db1d6c589c2eae45ad1eb862bc`
Windows x64 ZIP	218,134,349 bytes. SHA-256: `5977838d86aaa9e32f57d717728c593f0e04713905d2b6dfc5b5ed912dba79f7`

Intel was tested through Rosetta, not physical Intel hardware. Real cameras, boards, pointer mechanics, APRS radio and LTU network were not connected. Automated checks do not establish layout at every display scale, field performance, notarization or qualified controlled-flight dynamics.

30. Extension and maintenance guidance

30.1. Hardware and new telemetry

Specify framing, CRC, vehicle/source IDs, units, axes, timestamps/reset behavior and channel mapping before adding an adapter. Populate explicit measured/demand fields only when supplied. Preserve the Zephyrus adapter for old hardware/logs. Add captured packets, malformed/reboot tests, source isolation, recording/archive round trips and UI availability checks. Canard/tab names must not be assigned speculatively to legacy channels. Individual-servo commands require a contract; current encoders expose grouped roll/airbrake controls.

Define Launch enable and Iris selector commands, acknowledgements and timeout semantics before replacing placeholders. Separate desired from confirmed state; uncertain partial writes remain nonretrying. Measured pointer feedback needs a structured parser distinct from last-sent target. Homing, cable limits and stop must have firmware support rather than merely a renamed UI button.

30.2. Data, simulation and presets

Add presets to `site_presets.py` and the corresponding location UI, keeping canonical coordinates, exact codes and explicit elevations. Test wizard defaults, format changes, manual edits and persistence. Optional schema fields need defaults and validation across mission/session/flight. Breaking semantics need a version bump and migration/rejection, not silent unknown-field filtering.

Preserve archive limits, atomic replacement, active snapshot prefixes and source provenance. Changes to the Java bridge require rebuilding against the matching engine and recording options/input hashes. Controlled dynamics require model qualification; do not clear `controlled_model_validated:false` merely to suppress a label. Preserve raw inputs alongside any future fused estimate.

30.3. Known follow-up work

The release retains APRS, legacy terminology where it identifies the protocol, and two Launch displays. Requested future changes include Launch-only YouTube streaming, compact Windows layout improvements, a one-window/individual-servo workflow, removal of APRS/legacy-facing labels, and a larger Away vertical plot. These requests are not implemented by this documentation revision. Protocol work for station transport, switches and sensor fusion remains separate.

For each change, check role/source boundaries and ownership, run meaningful focused tests then release checks, inspect native windows at intended sizes and representative smaller/high-DPI settings, rebuild the target package and update operator/developer references. Never use physical recovery actuation as a software smoke test.

A. Complete mission configuration reference

The table follows the current `Mission` dataclass, including retained compatibility fields. Mission JSON adds `schema_version:1`; the default values below are application defaults, not a recommended launch configuration. The validator applies finite-number/range checks where defined; UI spinbox limits and dataclass validation are not identical for every field.

Field	Default	Meaning
`name`	Untitled mission	Human-readable mission name.
`latitude`	0.0	WGS84 launch latitude in degrees; -90 to 90.
`longitude`	0.0	WGS84 launch longitude in degrees; -180 to 180.
`altitude`	0.0	Launch ellipsoid altitude in metres; used by geographic transforms.
`altitude_msl`	0.0	Launch mean-sea-level elevation in metres; weather and engine atmosphere.
`site_configured`	False	Explicit origin-established flag; required for LIVE trajectory and simulation.
`launch_location_format`	latlon	Entry provenance: latlon, mgrs or pluscode.
`launch_location_code`	(empty)	Entered/resolved code; up to 80 characters. Canonical lat/lon remains authoritative.
`launch_site_name`	(empty)	Preset name, currently URRG or empty for Custom; up to 80 characters.
`legacy_altitude`	unknown	unknown, ellipsoid, gps_agl or barometric_agl; selects trajectory height interpretation.
`canard_count`	4	Number of UI canard slots, integer 0 to 16; four tabs are always added.
`pointer_latitude`	0.0	Virtual antenna WGS84 latitude, degrees; physical tracking retains frozen receiver GPS.
`pointer_longitude`	0.0	Virtual antenna WGS84 longitude, degrees.
`pointer_altitude`	0.0	Virtual antenna WGS84 ellipsoid altitude, metres; physical legacy tracking retains height zero.
`pointer_site_configured`	False	Boolean assertion that the antenna location is established; required for virtual trajectory tracking.
`pointer_location_format`	latlon	Antenna entry: latlon, mgrs or relative; relative coordinates are recalculated during validation.
`pointer_location_code`	(empty)	MGRS entry provenance, up to 80 characters; canonical coordinates are authoritative for absolute entry.
`pointer_site_name`	(empty)	Named URRG station preset, or empty for custom coordinates; does not supply elevation.
`pointer_launch_heading`	0.0	True heading from antenna to rocket on its launch pad, 0 to 360 degrees.
`pointer_launch_distance`	0.0	Horizontal antenna-to-pad range in the antenna ENU frame, 0 to 100000 metres.
`pointer_height_difference`	0.0	Antenna minus launch ellipsoid altitude, metres; positive means antenna above launch site.
`pointer_calibrated`	False	Retained assertion, always cleared by disk load and mode changes; not a manual-send prerequisite.
`pointer_az_offset`	0.0	Retained offset field; not applied by current legacy manual/tracking dispatch.
`pointer_el_offset`	0.0	Retained offset field; not applied by current legacy manual/tracking dispatch.
`pointer_el_min`	0.0	Stored route-helper envelope minimum; not an enforced operator control limit.
`pointer_el_max`	80.0	Stored route-helper envelope maximum; ordered within -90 to 90 degrees.
`pointer_az_min`	0.0	Stored route-helper azimuth minimum; not enforced by the legacy dispatch path.
`pointer_az_max`	359.0	Stored route-helper azimuth maximum; ordered within 0 to 360 degrees.
`pointer_full_rotation`	False	Stored route-helper choice; not a firmware cable-wrap guarantee.

Field	Default	Meaning
freshness	2.0	Maximum acceptable sample age, 0.1 to 60 seconds.
video_offset	0.0	Seconds; positive value delays recorded video relative to telemetry.
wind	One calm layer at height 0	1 to 200 increasing nonnegative AGL heights with east-/north velocity components; max speed 150 m/s.
wind_source	Manual · calm	Human-readable profile provenance.
weather_raw	{}	Cached normalized request/result/provider payload, when retrieved.
model	(empty)	Selected ORK path. Archives embed it and rebind the extracted path.
motor_files	[]	Up to 100 custom curve paths; selected files are embedded in flight archives.
simulation_index	0	Saved ORK simulation, zero-based integer 0 to 1000.
rail_length	2.0	Launch rod length in metres; greater than 0 and at most 100.
rail_tilt	0.0	Tilt from vertical in degrees; at least 0 and less than 90.
rail_heading	0.0	True-north bearing in degrees; UI range 0 to 359.99.
seed	42	Integer random seed, 0 to 2147483647.
low_battery	9.0	Total-battery alert threshold in volts; 1-second dwell and 0.3 V clearing hysteresis.

B. Complete normalized sample reference

`Sample.from_dict` retains known dataclass fields and uses defaults for absent optional values. The file formats add their own structural validation; the dataclass constructor alone is not a complete untrusted-input validator. Vector and dictionary meanings are defined by the producing protocol adapter.

Field	Default	Meaning
t	required	Device uptime seconds after wrap extension, or imported equivalent; align separately to flight zero.
sequence	required	Packet counter; LIVE wire counter is uint16.
source	required	Provenance such as LIVE, DEMO or LEGACY_CSV. REPLAY is a controller mode, not a rewrite of sample source.
received	runtime factory	Host monotonic receive time in seconds; replay refreshes this for current display freshness.
utc	runtime factory	Unix reception timestamp in seconds; original UTC retained for imported/demo data.
altitude	None	Reported filtered barometric altitude, metres, or unavailable.
velocity	None	Reported integrated velocity, metres/second, or unavailable.
latitude	None	Rocket WGS84 latitude in degrees, or unavailable.
longitude	None	Rocket WGS84 longitude in degrees, or unavailable.
gps_altitude	None	Reported GPS height in metres, with explicit mission interpretation.
gps_fix	0	GPS fix code; geographic display requires at least 3.
attitude	None	Optional three-element roll/pitch/yaw body-gyro integrals in degrees for Zephyrus; NOT Euler attitude. Valid orientation belongs in details.orientation.
rates	None	Optional XYZ angular rates, degrees/second; legacy Y sign is retained.
acceleration	None	Optional XYZ acceleration, metres/second squared.
enu	None	Optional local east/north/up metres; frame/provenance must be known.
battery	None	Optional total battery volts, sum of the three cells for LIVE.
rssi	None	Optional receiver RSSI in dBm.
phase	Unknown	Received or imported state label; unknown codes remain visible.

Field	Default	Meaning
`actuators`	runtime factory	Dictionary by channel name; demand/measured angles or legacy drive when actually available.
`details`	runtime factory	Extensible decoded legacy values, boot/time metadata, receiver data, raw CSV provenance and quality notes.

C. Legacy CSV columns and derived values

This is the exact 43-column order written by the compatibility adapter. The rich `SessionReader.export_csv` format is different and includes full sample JSON. Imported records retain their original row text and available decoded values. List padding described below refers to live good-packet output; it should not be used to invent extra hardware channels.

Column	Field	Meaning
1	`timestamp`	Receive UTC, Unix seconds.
2	`pyros`	Six pyro continuity codes.
3	`servos`	Four legacy drive values.
4	`servos_deg`	Four converted servo angles; first two use 60-degree scale, last two 50-degree scale; good CSV rounded to two decimals.
5	`accelerometer`	XYZ acceleration, m/s squared.
6	`barofilteredalt`	Filtered barometric altitude, m.
7	`temp`	Legacy calibrated temperature, degrees C.
8	`gyro`	XYZ gyro rates, degrees/s.
9	`gps_fix`	Rocket fix code.
10	`lat`	Rocket latitude, degrees.
11	`lon`	Rocket longitude, degrees.
12	`gpsalt`	GPS height, m; datum depends on firmware/configuration.
13	`gps_horiz_prec`	Horizontal GPS accuracy, m.
14	`gps_vert_prec`	Vertical GPS accuracy, m.
15	`gps_num_sat`	Satellite count.
16	`flight_time`	Original raw device clock, milliseconds.
17	`yaw_gyro_int`	Integrated yaw, degrees.
18	`pitch_gyro_int`	Integrated pitch, degrees.
19	`roll_gyro_int`	Integrated roll, degrees.
20	`state`	Legacy state enum string.
21	`pktnum`	Packet counter.
22	`rssi`	Ground receiver RSSI, dBm.
23	`armed_pyros`	Armed bits as list; good-packet CSV retains two unused zero entries.
24	`fired_pyros`	Fired bits as list; same good-packet padding.
25	`badpackets`	Rejected-packet counter.
26	`rxrssi`	Onboard RSSI, dBm.
27	`accel_integrated_velo`	Reported integrated velocity, m/s.
28	`baro_max_alt`	Maximum barometric altitude, m.
29	`gps_max_alt`	Maximum GPS altitude, m.
30	`pyro_resistances`	Pyro resistance list; good-packet output pads two unused zero entries.
31	`cell_voltages`	Three cell voltages, V.
32	`total_current`	Total current, A.
33	`converter_voltages`	Six rail voltages, V.
34	`converter_currents`	Six rail currents, A.
35	`bms_protections_enabled`	BMS enabled protection bits.
36	`bms_protection_status`	BMS triggered protection bits.
37	`bms_temp`	BMS temperature, degrees C.
38	`enabled_status`	Derived rail-enabled states from voltage proximity to nominal.

Column	Field	Meaning
39	`angleFromVertical`	Legacy tilt proxy derived from gyro-integral pitch/yaw, degrees; not measured tilt.
40	`gnd_lat`	Frozen/display receiver latitude convention, rounded to five decimals.
41	`gnd_lon`	Receiver longitude convention, rounded to five decimals.
42	`gnd_fix`	Legacy ground fix Boolean: trailer fix equals 3.
43	`gnd_alt`	Legacy receiver height: signed wire integer divided by 1000, rounded for display/CSV.

The live servo degree conversion is

$$\alpha_i = \frac{d_i - 1500}{500} \begin{cases} 60^\circ, & i = 0, 1, \\ 50^\circ, & i = 2, 3. \end{cases}$$

The legacy tilt proxy (not a measured attitude) is $\arccos(\cos p \cos y)$, with the cosine product clamped to $[-1, 1]$. The raw temperature calibration in `legacy_values` is

$$T = \frac{2000 + (T_{\text{raw}} - \texttt{0x91E3} \cdot 256)\texttt{0x6FEC}/2^{23}}{100}.$$

These formulas reproduce the former monitor; they do not establish new sensor calibration accuracy.

D. Worked file examples

The `examples` directory accompanies the source report. Its inputs have been loaded through the application's validators as documentation examples.

D.1. URRG planning template

`examples/urrg-mission.json` includes all current mission fields and the exact URRG code/canonical coordinates. It deliberately leaves `site_configured:false`, altitude convention unknown and both elevations zero so the example cannot be mistaken for a surveyed setup. Open it with **Open mission**, then review and complete those values. The essential location portion is:

```
{
  "schema_version": 1,
  "name": "URRG template - verify launch elevations",
  "launch_site_name": "URRG",
  "launch_location_format": "mgrs",
  "launch_location_code": "18TUN2061530290",
  "site_configured": false,
  "legacy_altitude": "unknown"
}
```

The full file also supplies the decoded latitude and longitude; do not treat this shortened excerpt as a replacement for the full template.

D.2. Wind profile

```
{
  "schema_version": 1,
  "source": "Documentation example only",
  "layers": [
    {"height": 0, "east": 0, "north": 0},
    {"height": 1000, "east": 3, "north": -1}
  ]
}
```

The second layer has eastward and southward components. This is a formatting example, not a retrieved URRG forecast.

D.3. Normalized trajectory

```
time_s,east_m,north_m,up_m
0,0,0,0
1,1,2,10
2,2,4,20
```

The matching manifest is:

```
{
  "schema_version": 1,
  "frame": "ENU",
  "units": "m,s",
  "name": "Documentation geometry example",
  "altitude_datum": "launch_relative",
  "origin": [
    42.70416776971639,
    -77.19029501648315,
    0
  ],
  "synthetic": true,
  "controlled_model_validated": false
}
```

The example is explicitly synthetic and is therefore rejected as a LIVE reference. It illustrates file structure and can be used for offline tests.

D.4. Portable flight metadata excerpt

```
{
  "format": "rocket-gnc-flight",
  "schema_version": 1,
  "app_version": "0.1.1",
  "source_mode": "DEMO",
  "scope": "Complete Zephyrus GS2 dataset",
  "session": "session",
  "samples": 5464,
  "position": 174.0,
  "flight_zero": 747.766,
  "time_aligned": true,
  "files": {
    "mission.json": {
      "size": "<actual byte count>",
      "sha256": "<actual content hash>"
    }
  }
}
```

This explanatory excerpt omits other entries and contains descriptive placeholders; it is not an importable archive. Use the app's Save flight operation to create a valid inventory and package.

E. Startup preferences example

The optional startup-choice file contains the four profile fields below. Its existence represents opt-in remembering; there is no separate remember Boolean inside the profile. Prefer the wizard/CLI to edit this file.

```
{
  "station": "away3",
  "role": "telemetry",
  "vehicle": "iris",
  "launch_site": "urrg"
}
```

The internal Launch key remains `base` for compatibility, while public UI uses Launch station. Supported station keys are `base`, `away1` through `away4`; roles are telemetry/video, vehicles balius/iris and sites custom/urrg. This preference is machine-local and is not part of mission JSON.

F. Source and validation index

Source links are pinned to the published v0.1.1 tag. Documentation generation extracts that commit into a temporary directory and ignores current checkout edits; the application checkout is not modified. Schema tables assert exact field-name coverage, and examples are loaded by the release's own validators. The report compiles without an application checkout using these frozen tables.

Source	Role
docs/V0A_STATION_MODES.md	Station/role layouts, receiver assignments, wizard persistence, network and placeholder boundaries.
docs/OPERATOR_GUIDE.md	Operator behavior and file/session workflows.
docs/DEVELOPER.md	Maintained standalone architecture and extension seams.
docs/PROTOCOL.md	Audited downlink/pointer/uplink wire contracts.
docs/LEGACY_PARITY.md	Original values, commands, shortcuts and compatibility fixes.
docs/DEMO_DATA.md	GS1/GS2/GS3 provenance, timing and quality limitations.
docs/BUILD.md, docs/MAC_UNIVERSAL.md	Native build/runtime preparation and universal composition.
docs/MIGRATION_VALIDATION.md	Exact October 25 source/package checks and environment limits.
docs/THIRD_PARTY_NOTICES.md	Dependency notices and resource provenance.
resources/fonts/README.md	Packaged user-designated font provenance.
resources/demo/manifest.json	Uncompressed recording hashes, row counts and timing.
resources/motors/manifest.json	Bundled motor integrity and identity.
tests/fixtures/legacy_reference.json	Captured original behavior and source hashes.
evidence/source-manifest.json	Every tracked file in the reviewed application commit, with size and SHA-256.
evidence/reference-validation.json	42 mission fields, 20 sample fields, 43 CSV columns and validated examples.
evidence/documentation-validation.json	Final PDF layout/content/font/figure verification, separate from application tests.

The existing v0 report and old planning documents remain historical records. No old screenshots, generated application sessions, runtimes or package binaries are copied into this report. The current application is maintained separately; this folder is a report source/PDF deliverable in the parent project repository.

G. Glossary

Term	Meaning in this app
AGL	Height above the configured launch/ground reference.
Ellipsoid height	Height relative to the WGS84 ellipsoid; used in ECEF/ENU transforms.
MSL	Mean sea level elevation; entered separately for atmosphere and weather.
ECEF / ENU	Earth-centered Earth-fixed / local east-north-up coordinate systems.
GNC	Guidance, navigation and control; this app monitors available data and sends legacy commands.
MGRS / Plus Code	Offline geographic coordinate representations; neither supplies elevation.
Mission	Configuration object; mission JSON alone can contain external asset paths.
Session	Directory containing indexed recorded data, raw bytes, CSVs, events and optional video/reference.
Flight file	Portable versioned ZIP64 archive with mission/assets and available recording/reference data.
Snapshot	Consistent committed subset saved while acquisition continues; active video segment may be absent.
Nominal reference	OpenRocket output under the recorded options; not qualified controlled-vehicle dynamics.
Sent / acknowledged	Serial dispatch / confirmed device response. The legacy command paths provide the first, not the second.
Source mode / phase	LIVE, DEMO or REPLAY / received onboard rocket state; independent concepts.
WAL	SQLite write-ahead log; retain it with a crashed source session during recovery.